UNIDAD 2: Gestión Lean Agile de Productos de Software

Tema 2: Filosofía Lean y método Kanban aplicados al desarrollo de software

---

1. Definición, Orígenes y Filosofía Central

Filosofía Lean

Nace en la década de 1940 en Japón con la industria automotriz (Toyota Production System - TPS, de la mano de Taiichi Ohno y Shigeo Shingo). Su principio rector es **maximizar el valor generado para el cliente reduciendo o eliminando sistemáticamente el desperdicio (*waste*)**.

En el ámbito del software, Mary y Tom Poppendieck (*Lean Software Development: An Agile Toolkit*, 2003) traducen la manufactura del TPS al trabajo intelectual de desarrollo de sistemas.

- **Definición de Valor**: Crear beneficio real para el cliente/usuario final.
- **Definición de Desperdicio (*Muda*)**: Cualquier actividad, artefacto, demora o esfuerzo consumido en el proceso que no añade valor directo al producto final desde la perspectiva del cliente.
- **Principio rector de productividad**: No se mide en cuántas líneas de código o artefactos se construyen por día, sino en descubrir e implementar sistemáticamente **lo correcto para construir cada día**.

Método Kanban

Desarrollado para el trabajo del conocimiento e ingeniería de software por **David J. Anderson** (2004-2007 en Microsoft y Corbis).

- **Definición**: Es un **método de gestión del flujo de trabajo y de cambio evolutivo/incremental** que utiliza un sistema visual de señalización mediante tarjetas (*kan-ban*) y tracción (*pull*) para optimizar la entrega de servicios de conocimiento.
- **Aclaración clave de examen**: **Kanban NO es un ciclo de vida de desarrollo de software ni una metodología de gestión de proyectos**. Es una herramienta de gestión sobre un proceso ya existente (*"comienza con lo que haces ahora"*), diseñada para catalizar mejoras continuas con mínima resistencia al cambio.

---

2. Los 7 Principios del Desarrollo Lean de Software (Poppendieck)

El matrimonio Poppendieck adaptó los principios industriales fijando 7 pilares fundamentales para el software:

1. **Eliminar desperdicios (*Eliminate Waste*)**:
   - Es el principio central. Exige eliminar cualquier paso, documento o código que no agregue valor directo.
2. **Amplificar el aprendizaje (*Amplify Learning*)**:
   - El desarrollo de software es un proceso continuo de descubrimiento de conocimiento. Promueve ciclos breves de retroalimentación, refactorización y pruebas frecuentes para aprender rápido (*build-measure-learn*).
3. **Diferir compromisos / Tomar decisiones tardías (*Decide as late as possible / Defer commitment*)**:
   - Retrasar las decisiones irreversibles hasta el **último momento responsable** (*last responsible moment*), reduciendo la incertidumbre y evitando trabajar sobre suposiciones prematuras que luego cambian.
4. **Entregar lo antes posible (*Deliver fast*)**:
   - Acotar los ciclos de desarrollo para entregar pequeños incrementos funcionales al cliente antes de que sus necesidades de negocio o el mercado cambien.
5. **Empoderar / Dar poder al equipo (*Empower the team*)**:
   - Los desarrolladores y especialistas que están en el detalle técnico deben tener la autoridad para tomar decisiones de diseño y proceso, promoviendo la automotivación.
6. **Construir la integridad conceptual (*Build integrity in*)**:
   - Asegurar la calidad intrínseca del software desde el inicio (mediante arquitectura limpia, pruebas automatizadas y refactorización) para evitar la degeneración del sistema.
7. **Ver el todo / Optimizar el sistema completo (*Optimize the whole*)**:
   - Evitar la suboptimización local (por ejemplo, optimizar solo la etapa de desarrollo si el embotellamiento real está en el despliegue o en la definición del negocio).

3. Los 7 Desperdicios (*Muda*) en el Desarrollo de Software

Shigeo Shingo clasificó los desperdicios en manufactura. Poppendieck los tradujo directamente al desarrollo de software en la siguiente tabla comparativa:

Desperdicio en Manufactura	Desperdicio en Software (Poppendieck / UTN)	Manifestación y Problemas en Proyectos de Software
Inventario	**Trabajo parcialmente terminado** (*Partially done work*)	Código no probado, historias a medio codificar, especificaciones no implementadas. Acumula riesgo e incertidumbre.
Sobreproducción	**Características extra** (*Extra features / Featuritis*)	Funcionalidades que el cliente no pidió o rara vez utiliza (el 80% del uso se concentra en el 20% de las funciones - Regla de Pareto).
Proceso extra	**Re-aprendizaje / Proceso extra** (*Relearning / Extra processing*)	Documentación burocrática no leída, reaprender lo que ya se sabía por falta de comunicación o rotación de personal.
Transporte	**Traspasos / Transferencias** (*Handoffs*)	Pasar documentos entre áreas separadas (ej. de analistas a desarrolladores, y de estos a testers), lo que degrada la información recibida.
Movimiento	**Cambio de tareas / Multitasking** (*Task switching*)	Interrupciones y cambio de contexto en el trabajo intelectual; requiere alto tiempo de aclimatación o seteo mental.
Esperas	**Demoras / Retrasos** (*Delays*)	Esperar aprobaciones de la gerencia, disponibilidad de servidores, decisiones de diseño o respuestas del cliente.
Defectos	**Defectos / Retrabajo** (*Defects*)	Bugs detectados tardíamente que obligan a rehacer código y romper el flujo de trabajo.
(+1 Agregado)	**Talento no utilizado** (*Unused talent*)	No aprovechar la creatividad, experiencia o habilidades técnicas de los miembros del equipo.

---

4. Método Kanban: Principios y Prácticas Generales

Kanban se basa en el **mecanismo de tracción (*Pull System*)**: los miembros del equipo "jalan" una nueva tarea solo cuando hay capacidad disponible, en oposición al sistema tradicional de empuje (*Push System*), donde se asigna trabajo saturando la capacidad.

Las 6 Prácticas Generales de Kanban (David J. Anderson):

1. **Visualizar el flujo de trabajo**:
    - Representar el flujo de valor mediante un **Tablero Kanban** dividido en columnas (ej. *Backlog, Análisis, Desarrollo, Testing, Despliegue*) y utilizar tarjetas (*kan-bans*) para representar las piezas de trabajo (US, bugs, tareas).
2. **Limitar el Trabajo en Curso (WIP Limits - *Work In Progress*)**:
    - Asignar un número máximo permitido de tarjetas para cada columna/estado del tablero.
    - *Fundamento Teórico*: Basado en la **Ley de Little** ($LeadTime \approx \frac{WIP}{Throughput}$). Para reducir el tiempo de entrega (*Lead Time*), se debe limitar estrictamente el trabajo en proceso (*WIP*).
3. **Gestionar el flujo**:
    - Observar y monitorear el movimiento del trabajo a través de las columnas, identificando cuellos de botella y bloqueos para mantener un flujo continuo y predecible.
4. **Hacer las políticas explícitas**:
    - Publicar las reglas del proceso de forma clara en el tablero: criterios para traccionar un ítem a la siguiente columna (Definition of Done de cada etapa), límites WIP y reglas de reposición.
5. **Implementar ciclos de retroalimentación (Cadencias)**:
    - Establecer reuniones periódicas de coordinación y revisión a distintos niveles:
        - *Team Kanban Meeting* (Diaria): Sincronización sobre el flujo del trabajo y desbloqueo de tarjetas.
        - *Replenishment Meeting* (Semanal/A demanda): Selección y priorización de ítems para ingresar al tablero.

        - *Retrospective* (Quincenal/Mensual): Análisis de la eficacia del proceso y del equipo.
6. **Mejorar colaborativamente, evolucionar experimentalmente**:
    - Promover una cultura de mejora continua (*Kaizen*) aplicando el método científico y experimentos seguros frente a fallas (*safe-to-fail experiments*).

---

5. Clases de Servicio y Cadencias Desacopladas

- **Clases de Servicio**: Permiten gestionar ítems con diferente urgencia e impacto financiero mediante reglas diferenciadas (políticas explícitas):
    1. *Expedite (Urgente)*: Atraviesa el tablero inmediatamente ante fallas críticas en producción, superando temporalmente los límites WIP.
    2. *Fixed Date (Fecha Fija)*: Fechas límite estrictas por motivos legales o comerciales.
    3. *Standard (Estándar)*: Flujo habitual de funcionalidades.
    4. *Intangible*: Mantenimiento o deuda técnica sin fecha límite rígida pero con valor a largo plazo.
- **Cadencias Desacopladas**: A diferencia de Scrum (donde planificación, desarrollo y entrega ocurren dentro de la misma ventana fija o *timebox*), Kanban **desacopla las cadencias**: la cadencia de priorización/planificación, la cadencia de desarrollo y la cadencia de despliegue a producción pueden ocurrir con frecuencias independientes según las necesidades del negocio.

---

6. Cuadro Comparativo: Scrum vs. Kanban (Pregunta Típica de Examen)

Criterio / Dimensión	Scrum	Kanban
Unidad de tiempo / Ritmo	Cadencia fija en iteraciones rígidas (*time-boxed Sprints*, 1 a 4 semanas).	Flujo continuo. Cadencias desacopladas y no obligatoriamente repetitivas.
Limitación del WIP	**Indirecta**: Se limita el trabajo al inicio del Sprint mediante la capacidad del equipo.	**Directa**: Se fijan límites explícitos de WIP por estado/columna en el tablero.
Roles Prescritos	Obligatorios (Product Owner, Scrum Master, Developers).	**Ninguno prescrito**. Se aplica sobre los roles organizacionales existentes.
Cambios de alcance durante la ejecución	No se permiten cambios en el Sprint Backlog una vez iniciado el Sprint.	Se permiten cambios continuos siempre que exista capacidad disponible según el WIP.
Métrica principal de planificación	**Velocidad** (Story Points completados por Sprint).	**Lead Time / Cycle Time** (tiempo promedio en atravesar el sistema).
Permanencia del Tablero	Se reinicia/limpia al finalizar cada Sprint.	**Persistente**. Es permanente y el trabajo fluye de manera continua.
Estimación de Tareas	Obligatoria en la planificación del Sprint.	Opcional (se busca la homogeneidad de tamaño o análisis de flujo).

---

7. Métricas en Kanban (Introducción a la Medición de Flujo)

El foco de medición en Kanban es el **comportamiento del proceso a lo largo del tiempo**:

1. **Lead Time (Tiempo de Entrega)**: Transcurrido desde que el cliente solicita el ítem hasta su entrega final (perspectiva del cliente).
2. **Cycle Time (Tiempo de Ciclo)**: Transcurrido desde que el equipo inicia efectivamente el trabajo sobre el ítem hasta que se completa (perspectiva interna del equipo).
3. **Touch Time (Tiempo de Trabajo Efectivo)**: Tiempo neto en el que se estuvo trabajando activamente en el ítem, excluyendo bloqueos y esperas.
4. **Eficiencia del Ciclo de Proceso**: Calculado como $\frac{TouchTime}{LeadTime}$. Mide qué porcentaje del tiempo total representó trabajo real que agregó valor.

Tema 3: Métricas en los enfoques tradicional, Lean y Agile

---

1. Definición, Objetivos y Dominios de las Métricas de Software

Una **métrica de software** es una medida cuantitativa que representa el grado o presencia de un determinado conjunto de atributos respecto a un proceso, proyecto o producto de software. Su objetivo primordial no es controlar burocráticamente, sino **aportar visibilidad para tomar decisiones informadas y guiar la mejora continua**.

Los 3 Dominios de las Métricas (Cátedra UTN):

1. **Métricas de Proceso**: Miden la eficacia, madurez y desempeño del marco de trabajo a nivel organizacional. Permiten aprender del histórico organizativo para optimizar las actividades de ingeniería.
2. **Métricas de Proyecto**: Son métricas tácticas que permiten al líder o equipo gestionar las variables operativas (tiempo, esfuerzo, costo, riesgos y asignación de recursos) en tiempo real para cumplir con los compromisos.
3. **Métricas de Producto**: Miden los atributos y la calidad del software construido (tamaño, complejidad, densidad de defectos, mantenibilidad, cobertura de pruebas).

Nota de articulación: Las métricas recopiladas en los proyectos individuales se consolidan a lo largo del tiempo para generar métricas de proceso públicas para toda la organización.

---

2. Métricas en el Enfoque Tradicional (Basado en Procesos Definidos)

En la gestión tradicional o predictiva, la medición es una **disciplina transversal y formalizada**. Se define exhaustivamente en la etapa inicial de planificación (qué se va a medir, cómo se calcula, quién recolecta el dato y con qué frecuencia).

- **Orientación**: Apunta a controlar las desviaciones respecto del plan original, fundamentándose en la **Triple Restricción Tradicional** (Alcance fijo; Costo y Tiempo estimados).
- **Dependencia por Roles**: Diferentes métricas interesan a distintos perfiles: los perfiles técnicos miran tamaño, esfuerzo y defectos; mientras que los líderes y gerentes observan el calendario, costos e hitos del contrato.

Las 4 Métricas Básicas e Imprescindibles de Proyecto/Producto:

1. **Tamaño del Producto**: Medida cuantitativa del volumen del software. Históricamente medido en Líneas de Código (LOC), y modernamente en métricas funcionales como **Puntos de Función (FP)**, Requerimientos o Casos de Uso.
2. **Esfuerzo**: Cantidad de trabajo humano consumido, expresado en Horas-Hombre o Personas-Mes.
3. **Calendario / Tiempo**: Cumplimiento del cronograma, avance de hitos e intervalos del Diagrama de Gantt.
4. **Defectos**: Tasa de fallas encontradas durante las fases de verificación formal o pruebas de integración.

---

3. Métricas en Ambientes Ágiles (Procesos Empíricos)

En el marco ágil, la medición responde a la naturaleza de los **procesos empíricos** (basados en inspección y adaptación constante). Nace como una reacción crítica ante el enfoque tradicional, donde solían gastarse ingentes recursos midiendo procesos y proyectos sin garantizar avances reales en el código.

Principios y Reglas de Oro en Ágil:

- **Regla de Oro**: *"La medición es una salida, no una actividad"* → *"Medir lo que sea necesario y nada más"*.
- **Principio Rector**: *"El software funcionando es la principal medida de progreso"* (Manifiesto Ágil).
- **Métricas Locales No Extrapolables**: Las métricas pertenecen exclusivamente a un equipo en su contexto. **Está conceptualmente prohibido comparar o extrapolar la velocidad o métricas entre diferentes equipos**.

Métricas Clave en Scrum / Ágil:

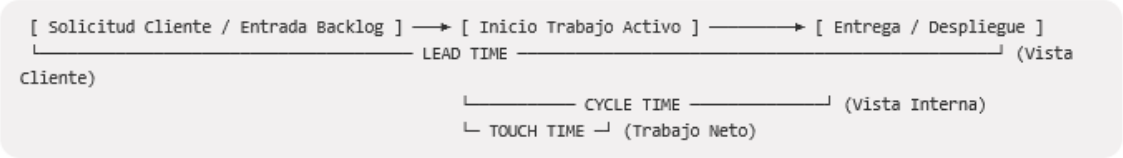
1. **Velocidad (*Velocity*)**:
    - **Definición**: Cantidad total de **Puntos de Historia (Story Points)** completados por el equipo durante un Sprint y **aceptados formalmente por el Product Owner**.
    - *Regla Estricta de Parcial*: **No se contabilizan historias de usuario parcialmente terminadas**; únicamente las que cumplen la Definición de Terminado (*DoD*) al 100% y fueron validadas.
    - *Uso*: Se representa en un gráfico de barras a lo largo de las iteraciones para analizar la **estabilidad del equipo** y predecir la capacidad de entrega futura.
2. **Capacidad (*Capacity*)**:
    - **Definición**: Cantidad total de disponibilidad real de trabajo que el equipo posee para el próximo Sprint, expresada en **horas ideales de trabajo**.
    - *Uso*: Herramienta de planificación utilizada durante el *Sprint Planning* para determinar cuántas US se pueden comprometer en base a las vacaciones, licencias o capacitaciones previstas.
3. **Funcionalidades Probadas en Ejecución (*Running Tested Features - RTF*)**:
    - **Definición**: Métrica de producto acumulativa que contabiliza el número de características/funcionalidades que no solo están codificadas, sino que han pasado con éxito sus pruebas de integración y aceptación automatizadas. Es el indicador más transparente de avance real.
4. **Diagramas de Flujo de Avance (*Burndown / Burnup Charts*)**:
    - *Sprint Burndown*: Muestra la cantidad de trabajo pendiente (en puntos o horas) día a día dentro del Sprint.
    - *Release Burnup*: Muestra la cantidad de trabajo completado acumulado hacia el Objetivo de Release frente al alcance total.

---

4. Métricas en el Enfoque Lean (Kanban - Flujo Continuo)

En Lean y Kanban no existe el concepto de iteraciones rígidas (*sprints*) ni de un "proyecto" con un inicio y fin prefijado; el foco se traslada a la **optimización del flujo de trabajo continuo y la eliminación de demoras**. Por ende, las métricas miden el comportamiento del proceso a través del tiempo.

Métricas Principales de Flujo en Lean/Kanban:



1. **Lead Time (Tiempo de Entrega / Elapsed Time)**:
    - **Definición**: Tiempo total transcurrido desde que el cliente realiza una solicitud (entra al Backlog) hasta que la funcionalidad está terminada y entregada en sus manos.
    - **Perspectiva**: **Vista del Cliente**. Es la métrica externa por excelencia para evaluar la satisfacción y la capacidad de respuesta de la organización.
2. **Cycle Time (Tiempo de Ciclo)**:
    - **Definición**: Tiempo transcurrido desde que el equipo de desarrollo toma activamente el ítem de trabajo (pasa a la columna "En Proceso" / *WIP*) hasta que se completa la fase de desarrollo/testing.
    - **Perspectiva**: **Vista Interna del Equipo**. Medición mecánica de la velocidad o ritmo de procesamiento del sistema de trabajo.
3. **Touch Time (Tiempo de Trabajo Efectivo)**:
    - **Definición**: Tiempo neto durante el cual un desarrollador o especialista estuvo trabajando de forma activa e interrumpida sobre la tarea, excluyendo tiempos de espera, bloqueos o colas de traspaso.
4. **Eficiencia del Proceso (*Process Efficiency*)**:
    - **Fórmula**: $Eficiencia del Proceso = \left(\frac{TouchTime}{LeadTime}\right) \times 100.$
    - Mide qué porcentaje del tiempo total que el requerimiento pasó en el sistema representó trabajo real agregando valor versus tiempo perdido en desperdicios (esperas y bloqueos).

---

5. Métricas para la Calidad del Producto de Software (Transversales)

Especialmente relevantes para evaluar la calidad técnica del código generado:

- **Cobertura de Pruebas (*Code Coverage*)**: Porcentaje de líneas o ramas de código ejecutadas por la suite de pruebas automatizadas.
- **Defectos por Severidad**: Clasificación de fallas según su impacto operacional (ej. *Bloqueante, Crítico, Mayor, Menor*).
- **Densidad de Defectos**: Cantidad de defectos detectados por unidad de tamaño (ej. *Defectos por Puntos de Historia* o *Defectos por KLOC*).

---

6. Tabla Comparativa General de Métricas entre Enfoques

Dimensión / Criterio	Enfoque Tradicional	Enfoque Ágil (Scrum)	Enfoque Lean (Kanban)
Objeto central de medición	Proceso, Proyecto y Producto de forma equilibrada.	**Producto funcionando** (resultado utilizable).	**Flujo continuo del proceso** y eliminación de cuellos de botella.
Métricas representativas	Esfuerzo, Tiempo/Calendario, Costos, Defectos, LOC, Puntos de Función.	Velocidad (SP), Capacidad (Horas), Running Tested Features (RTF).	Lead Time, Cycle Time, Touch Time, Eficiencia del Proceso.
Unidad de tiempo / Medida	Absoluta (Días, Meses, Horas-Hombre, Dinero).	Relativa / Timebox (Story Points por Sprint).	Tiempo continuo de flujo (Días hábiles por tarjeta).
Incentivo / Filosofía	Predictibilidad y cumplimiento del plan inicial.	Software funcionando y valor entregado al usuario.	Reducción del tiempo de ciclo y eliminación de desperdicios.
Extrapolación de datos	Fomentada a nivel organizacional mediante bases históricas.	**Prohibida**. La velocidad es local a cada equipo.	Analiza la capacidad y variabilidad del sistema propio.

---

7. Relaciones, Mitos y Trampas Típicas de Examen

- **Trampa 1: "La velocidad sirve para comparar qué equipo es más productivo"**: **FALSO**. Los Puntos de Historia son una estimación relativa propia de las dinámicas internas de cada equipo. Una velocidad de 30 SP en el Equipo A no significa que produzca más que una velocidad de 15 SP en el Equipo B.
- **Trampa 2: "Contabilizar Puntos de Historia parcialmente terminados al final del Sprint"**: **FALSO**. Si una US de 8 SP quedó al 90%, se suman **0 Puntos de Historia** a la velocidad del Sprint. Los 8 SP se reestiman y computarán en el Sprint donde la historia se termine al 100%.
- **Trampa 3: Confundir Lead Time con Cycle Time**: El **Lead Time** arranca desde que el cliente lo pide (vista externa); el **Cycle Time** arranca cuando el equipo empieza a construirlo (vista interna).
- **Trampa 4: "Lean y Ágil no miden nada"**: **FALSO**. Miden bajo una filosofía **minimalista** ("medir lo necesario y nada más"), concentrándose en el flujo de valor y en la calidad intrínseca del software entregado.

UNIDAD 3: Gestión de Software como Producto

Tema 4: Prácticas Continuas

---

1. Definición, Objetivos y Contexto en la Gestión de Producto

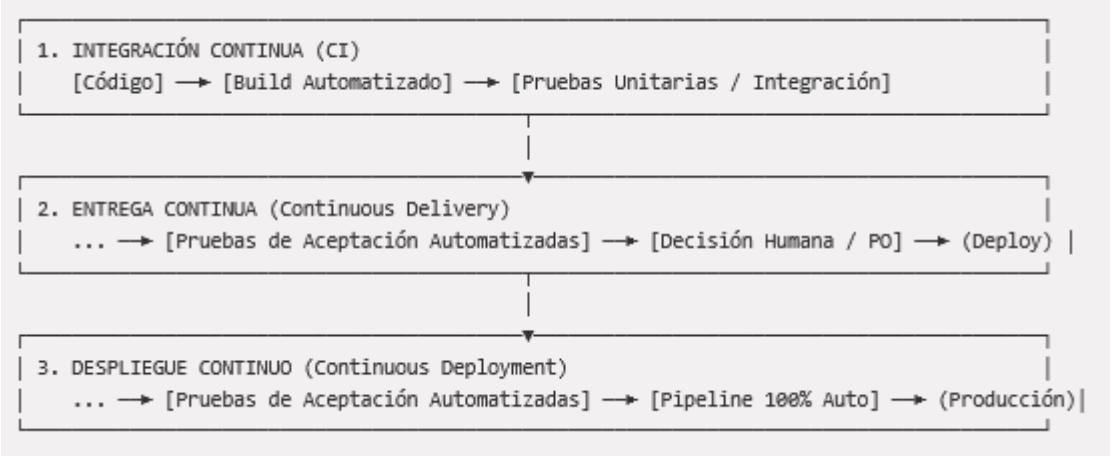
Las **prácticas continuas** son un conjunto de disciplinas técnicas y operativas propias del desarrollo moderno de software que buscan automatizar y acelerar el flujo que va desde la concepción de una idea hasta su disponibilización al usuario final.

En la gestión de software como producto, la SCM (Gestión de Configuración de Software) tradicional deja de ser una actividad burocrática de control para transformarse en un motor de soporte diario embebido en las actividades del equipo. Su objetivo central es **garantizar la integridad, calidad y trazabilidad del producto** manteniendo siempre una versión estable y lista para ser entregada.

---

2. La Trilogía de Prácticas Continuas (Flujo de Pipeline)

El ciclo de vida del código a través del servidor de integración y despliegue se divide en tres niveles evolutivos:



A. Integración Continua (Continuous Integration - CI)

- **Definición**: Práctica de desarrollo mediante la cual los desarrolladores integran las modificaciones de su código en un repositorio central compartido **varias veces al día**.
- **Mecanismo de funcionamiento**: Cada vez que un desarrollador realiza un *commit* o *push* a la línea principal (*main* o *trunk*), un servidor de CI (ej. Jenkins, GitHub Actions, Hudson) detecta el cambio de forma automática:
    - Descarga el código actualizado de la línea principal.
    - Ejecuta un **build automatizado** (compilación y empaquetado del sistema).
    - Corre la suite de **pruebas unitarias y de integración automatizadas**.
- **Objetivos y Beneficios**:
    - Detectar y corregir fallas de integración de manera temprana ("*fail fast*"), cuando el cambio de código aún es pequeño y fácil de rastrear en la memoria del desarrollador.
    - Eliminar definitivamente el problema clásico de *"en mi máquina funciona"* al validar los cambios en un entorno de integración neutral.
    - Mantener la rama principal en un estado técnico siempre compilable y libre de errores graves.

B. Entrega Continua (Continuous Delivery)

- **Definición**: Disciplina en la cual el software se construye y mantiene de forma tal que **puede ser liberado a producción en cualquier momento** mediante un proceso altamente automatizado.
- **Requisito previo**: La **Integración Continua es un prerrequisito obligatorio** de la Entrega Continua.
- **Mecanismo de funcionamiento**: Suma a la etapa de CI la **automatización de las pruebas de aceptación de usuario y de sistema**.
- **Característica Distintiva de Examen (El Componente Humano)**:
    - El producto queda técnicamente validado y en estado "desplegable" (*ready to deploy*).
    - **La puesta en producción NO ocurre de manera automática**: requiere la autorización o decisión explicita de un perfil de negocio (como el Product Owner o gerente de producto). Se ejecuta habitualmente presionando "un solo clic".

C. Despliegue Continuo (Continuous Deployment)

- **Definición**: Extensión de la entrega continua donde **todo cambio que supera con éxito la batería de pruebas automatizadas es liberado e instalado automáticamente en el entorno de producción** sin ninguna intervención humana.
- **Mecanismo de funcionamiento**: Se basa en *pipelines* (tuberías automatizadas de pasos ordenados) que coordinan la compilación, testing, empaquetado y transferencia a los servidores en producción.
- **Objetivo**: Minimizar el tiempo transcurrido entre la escritura de una línea de código y su uso real por parte del cliente, permitiendo múltiples despliegues diarios con bajo nivel de riesgo al empaquetar cambios pequeños.

---

3. Estrategias de Despliegue en Producción (Deployment Strategies)

Para lograr que el paso a producción en *Continuous Deployment* sea transparente y no interrumpa la disponibilidad del servicio (*zero downtime*), se aplican dos estrategias de arquitectura operativamente estándar:

Estrategia	Funcionamiento y Mecánica	Ventajas y Manejo de Riesgo
Canary Deployment	Consiste en realizar lanzamientos por etapas o fases. La nueva versión del producto se despliega primero para un **subconjunto muy reducido de usuarios** (p. ej. 5% del tráfico o usuarios de una región geográfica específica). Se monitorea el comportamiento y, si no surgen errores, se migra al resto de los usuarios de forma progresiva.	**Minimización de Impacto**: Si la nueva versión contiene un fallo no detectado, solo afecta a un número limitado de clientes antes de ser detectado y revertido.
Blue/Green Deployment	Utiliza dos entornos de producción idénticos: el entorno **Azul (Blue)** alberga la versión actualmente activa que recibe el 100% del tráfico de los usuarios; el entorno **Verde (Green)** alberga la nueva versión. Una vez que la versión Verde es probada en producción, el balanceador de carga o router conmuta instantáneamente el tráfico de Azul a Verde.	**Zero Downtime y Rollback Inmediato**: La conmutación de tráfico es prácticamente instantánea. Si se detecta una falla en el entorno Verde, se vuelve a enrutar el tráfico al entorno Azul (*rollback*) de inmediato sin perder tiempo reinstalando.

---

4. Prácticas de Ingeniería Ágil que Sostienen las Prácticas Continuas

Las prácticas continuas no pueden subsistir únicamente con herramientas de servidor; requieren hábitos técnicos rigurosos por parte del equipo de desarrollo:

1. **Desarrollo Conducido por Pruebas (Test Driven Development - TDD)**:
    - Ciclo iterativo "*Red-Green-Refactor*": 1) Escribir una prueba automatizada que falle (Red); 2) Escribir el código mínimo para que pase (Green); 3) Refactorizar para mejorar la estructura del código sin cambiar su comportamiento externo.
    - Garantiza que el código nazca con una suite de pruebas automatizadas que servirá de red de seguridad para el pipeline de CI.
2. **Refactorización (*Refactoring*)**:
    - Mejora continua del diseño interno del código para reducir su complejidad y deuda técnica.
3. **Gestión de Configuración y Control de Versiones en Ágil**:
    - **Uso de Repositorios**: Preferencia por repositorios distribuidos (como Git) donde cada commit se asocia a una tarea o Historia de Usuario.

- **Estrategia de Ramas Cortas (*Short-lived branches*)**: Trabajar en ramas de funcionalidad (*feature branches*) breves o directamente sobre el *trunk/main*, realizando fusiones (*merges*) frecuentes hacia la rama principal para evitar conflictos masivos de integración.
- **Etiquetado (*Tagging*)**: Marcar congelamientos o líneas base (*baselines*) en el repositorio para identificar formalmente qué commit exacto corresponde a cada *release* enviada a producción.

---

5. Cuadro Comparativo: CI vs. Continuous Delivery vs. Continuous Deployment

Dimensión	Integración Continua (CI)	Entrega Continua (CDel)	Despliegue Continuo (CDep)
Alcance del Pipeline	Código → Build → Pruebas Unitarias e Integración.	CI → Pruebas de Aceptación Automatizadas → Listo para Producción.	CI + CDel → Despliegue Automático a Producción.
Resultado final	Artefacto compilado y validado técnicamente.	Paquete en estado "Entregable" (*Deployable*) en staging/pre-producción.	Software instalado e impactando directamente en el usuario final.
Paso a Producción	No aplica (se limita al entorno de construcción/test).	**Manual / Decisión de Negocio** (aprobadador por el Product Owner).	**Totalmente Automatizado** (sin intervención humana).
Riesgo Operativo	Bajo (interno al equipo técnico).	Controlado (el negocio decide cuándo lanzar).	Requiere altísima madurez en automatización de testing y monitoreo.

---

6. Consideraciones de Examen y Confusiones Frecuentes

- **Diferencia Crítica de Parcial**: Confundir **Entrega Continua (*Delivery*)** con **Despliegue Continuo (*Deployment*)**.
  - En *Continuous Delivery*, el sistema queda listo para ser puesto en producción, pero la decisión de apretar el botón la toma un ser humano (el PO/Negocio).
  - En *Continuous Deployment*, no hay botón manual ni ser humano; el código que aprueba las pruebas viaja directamente al entorno de producción.
- **El Mito de la Herramienta**: Instalar Jenkins, GitLab CI o GitHub Actions no significa que un equipo haga Integración Continua. CI es una **disciplina de trabajo del desarrollador** (subir código al menos una vez al día con pruebas ejecutadas); la herramienta solo automatiza el proceso.
- **Relación con el Proceso Empírico**: Las prácticas continuas son el habilitador técnico imprescindible para sostener ciclos de vida iterativos e incrementales cortos. Sin CI/CD, las entregas frecuentes se vuelven insostenibles por el costo de las pruebas manuales y los despliegues complejos.

UNIDAD 4: Aseguramiento de Calidad de Proceso y de Producto

Tema 5: Conceptos generales sobre calidad

---

1. Definición Formal y Perspectivas de la Calidad

Definición General de Calidad

Según los estándares internacionales (ISO/ANSI) y la cátedra, la calidad es el **conjunto de aspectos y características de un producto o servicio que se relacionan con su habilidad de satisfacer necesidades manifiestas o implícitas**. Es un concepto esencialmente **relativo y subjetivo**: lo que para un usuario representa alta calidad, para otro puede ser insatisfactorio según el contexto, el momento, la edad y las necesidades específicas de uso.

Definición Formal de Calidad de Software (Pressman / Cátedra UTN)

En la ingeniería de software, la calidad se define formalmente como:

> *"El cumplimiento de los requerimientos funcionales y de performance explícitamente definidos, de los estándares de desarrollo explícitamente documentados y de las características implícitas esperadas del desarrollo de software profesional"*.

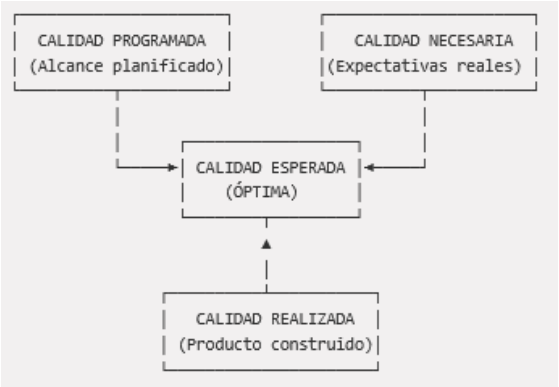
Los 3 Pilares Fundamentales de la Definición de Calidad de Software:

1. **Requerimientos explícitos**: Constituyen la base primaria de la medición. Si el software no cumple con los requerimientos acordados con el cliente, carece de calidad, sin importar cuán elegante sea su código.
2. **Estándares de desarrollo explícitos**: Si no se siguen estándares de codificación, diseño y documentación formalmente establecidos, el software presentará una baja calidad estructural que complicará su mantenimiento futuro.
3. **Requerimientos implícitos (expectativas profesionales)**: Existen atributos que el cliente rara vez menciona explícitamente (como la usabilidad, la seguridad, la fiabilidad o un tiempo de respuesta aceptable) pero que espera de un desarrollo profesional. Si se omiten, el producto será rechazado por los usuarios.

---

2. Las Tres Dimensiones de la Calidad en la Gestión

Para gestionar la calidad en las organizaciones de software es indispensable alinear tres perspectivas o esferas de coincidencia:



1. **Calidad Programada**: Los alcances, funcionalidades y especificaciones de calidad fijados en la planificación del proyecto.
2. **Calidad Realizada**: Lo que el equipo de ingeniería efectivamente construyó e implementó en el producto final.
3. **Calidad Necesaria**: Lo que el cliente y los usuarios realmente necesitan y esperan para resolver su problema de negocio.

- **El Equilibrio Óptimo (Calidad Esperada)**: Se alcanza únicamente en la intersección exacta de las tres esferas.

- **Impacto de las Desviaciones (Foco de Examen)**:
    - Todo lo que se programe o realice **por fuera de la calidad necesaria** constituye **desperdicio** (trabajo extra o funcionalidades que el cliente no usará).
    - Todo lo que la calidad realizada **no cubra de la calidad necesaria o programada** genera **insatisfacción, defectos y fallas de proyecto**.

---

3. Calidad de Producto vs. Calidad de Proceso

La gestión integral de la calidad en el software se divide operativamente en dos grandes enfoques complementarios:

A. Calidad de Producto

- **Foco**: Se concentra en el artefacto de software resultante (modelos, código fuente, ejecutables, bases de datos y manuales).
- **Objetivo**: Evaluar y medir si el producto entregado posee los atributos deseados (funcionalidad, mantenibilidad, velocidad, seguridad).
- **Mecanismos de Control**: Se realiza principalmente mediante **revisiones técnicas** y **actividades de pruebas de software (*testing*)**.

B. Calidad de Proceso

- **Foco**: Se concentra en el conjunto estructurado de actividades, métodos, prácticas, herramientas y transformaciones mediante los cuales las personas desarrollan y mantienen el software.
- **Objetivo**: Establecer y verificar que se cumplan los procedimientos y estándares organizacionales durante todo el ciclo de vida para prevenir la introducción de errores.
- **Mecanismos de Aseguramiento**: Se ejecuta mediante las actividades de **Aseguramiento de Calidad de Proceso y Producto (PPQA / SQA)** y **auditorías formales**.

El Principio Rector de la Cátedra (Relación Proceso-Producto):

> *"Se realiza un control de calidad sobre el proceso de desarrollo bajo el argumento de que este posee un gran impacto en la calidad del producto; es decir, en teoría la calidad del producto depende de la calidad del proceso utilizado"*.

- **Diferencia crucial entre manufactura e ingeniería de software**: En una fábrica de objetos físicos, si la máquina está bien calibrada (proceso estandarizado), todos los productos salen idénticos y con alta calidad. En software, el desarrollo **no es una manufactura mecánica, sino un proceso de diseño creativo e intelectual** donde influyen drásticamente el talento, la experiencia individual y la complejidad del negocio. Sin embargo, el **proceso es el único factor verdaderamente controlable** por la organización para mejorar sistemáticamente el rendimiento y reducir los defectos.

---

4. Estándares de Producto vs. Estándares de Proceso

El aseguramiento de la calidad requiere definir o seleccionar estándares explícitos que guíen al equipo:

- **Estándares de Producto**: Definen las características o formatos que todos los componentes y entregables del producto deben exhibir.
    - *Ejemplos*: Estándares de estilo y codificación (nombres de variables, estructura de clases), formatos para documentos de requerimientos, arquitectura de componentes.
- **Estándares de Proceso**: Definen los pasos, métodos y flujos de trabajo que deben ejecutarse obligatoriamente durante el desarrollo.
    - *Ejemplos*: Procedimiento para realizar revisiones de código, flujo de trabajo de Git (*branching strategy*), proceso para la gestión de cambios y liberaciones.

---

5. Introducción a los Modelos de Calidad de Producto

Para poder evaluar objetivamente la calidad de un producto de software, se utilizan modelos teóricos que estructuran sus atributos no funcionales en factores y subcaracterísticas:

1. **Modelo ISO/IEC 25010 (SQuaRE)**:
    - Define 8 características principales de calidad de producto: *Adecuación Funcional, Eficiencia de Desempeño, Compatibilidad, Usabilidad, Fiabilidad, Seguridad, Mantenibilidad y Portabilidad*.
2. **Modelo de McCall**:
    - Organiza la calidad en 3 ejes o agrupadores según la relación del usuario con el producto:
        - **Operación del Producto**: Corrección, Fiabilidad, Usabilidad, Integridad y Eficiencia.
        - **Revisión del Producto**: Facilidad de mantenimiento, Flexibilidad y Facilidad de prueba (*Testability*).
        - **Transición del Producto**: Portabilidad, Reusabilidad e Interoperabilidad.
3. **Modelo de Barbacci / SEI**:
    - Plantea que la calidad de producto surge del equilibrio global de tres dimensiones críticas contrapuestas: ***Performance*, Confiabilidad y Seguridad**. Optimizar al máximo una dimensión local (ej. la seguridad mediante cifrado pesado) puede degradar otra dimensión (ej. el rendimiento).

---

6. Cuadro Comparativo: Calidad de Proceso vs. Calidad de Producto

Criterio / Dimensión	Calidad de Proceso	Calidad de Producto
Objeto de evaluación	Las actividades, procedimientos, métodos y herramientas empleadas para construir el software.	Los entregables y artefactos de software generados (código, documentos, ejecutables).
Enfoque de gestión	**Preventivo**: Actúa antes y durante el desarrollo para evitar que se cometan errores.	**Detectivo / Correctivo**: Revela los defectos presentes en el producto construido.
Disciplinas asociadas	Aseguramiento de Calidad de Software (SQA / PPQA) y Auditorías de Proceso.	Control de Calidad, Testing (Pruebas de software) y Revisiones Técnicas.
Estándares de referencia	CMMI, ISO 9001, ISO/IEC 15504 (SPICE), procedimientos de trabajo.	ISO/IEC 25010, McCall, FURPS, estándares de codificación y diseño.
Responsables directos	Grupo o Área de Aseguramiento de Calidad (SQA/PPQA) y Líderes de Proceso.	Equipo de Desarrollo de Software y Equipo de Testing / Control de Calidad.

---

7. Relaciones, Confusiones y Trampas Frecuentes de Examen

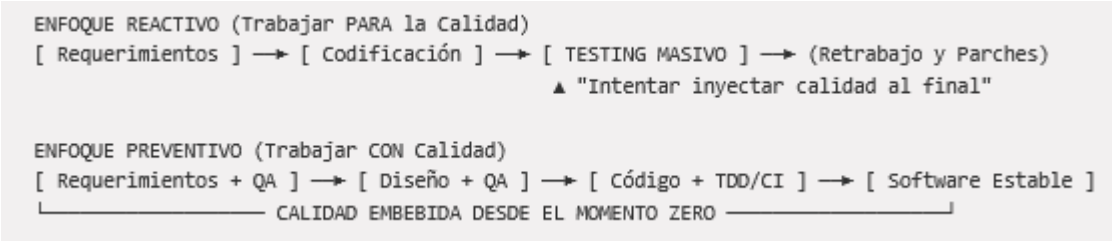
- **Trampa 1: Confundir Aseguramiento de Calidad (QA) con Control de Calidad (Testing)**:
    - **QA (Aseguramiento de Calidad)** se enfoca en el **proceso** y es preventivo.
    - **Testing (Control de Calidad)** se enfoca en el **producto** y es detectivo (llega cuando el código ya está escrito para buscar fallas).
- **Trampa 2: "Un proceso maduro garantiza un producto sin ningún defecto"**: **FALSO**. En software, un buen proceso disminuye drásticamente la densidad de defectos, pero debido a la naturaleza subjetiva y creativa del diseño, los factores humanos y los cambios de contexto del negocio, no existe el riesgo cero.
- **Trampa 3: "La calidad consiste en agregar la mayor cantidad de funcionalidades posibles al software"**: **FALSO**. Construir características que el cliente no pidió o no necesita atenta contra la calidad programada e introduce desperdicio y complejidad innecesaria.

Tema 6: Importancia de trabajar para y con calidad: ventajas y desventajas

---

1. Definición y Cambio de Paradigma: Trabajar "PARA" la Calidad vs. Trabajar "CON" Calidad

Dentro de la gestión de la calidad de software, existe una distinción filosófica y operativa fundamental que define el éxito o fracaso de los proyectos:



A. Trabajar "PARA" la Calidad (Enfoque Reactivo / Detección Tardía)

- **Concepto**: Consiste en desarrollar el software con baja o nula disciplina preventiva y confiar en que la calidad se logrará mediante una fase masiva e intensiva de pruebas (*testing*) o inspecciones al final del ciclo de desarrollo.
- **El Error Conceptual (Foco de Examen)**: Se basa en la falsa creencia de que la calidad puede "inyectarse" o agregarse al producto al momento de entregarlo.
- **Consecuencias**: Genera un ciclo interminable de corrección de fallas, retrabajo destructivo (*ping-pong* entre desarrollo y testing), atrasos severos en las entregas y costos fuera de control.

B. Trabajar "CON" Calidad (Enfoque Preventivo / Calidad Embebida)

- **Concepto**: Es el principio rector de la ingeniería de software moderna. Exige concebir, diseñar y construir el software aplicando prácticas de aseguramiento de calidad (*QA / PPQA*) de forma continua desde el momento cero (desde la captura de requerimientos hasta la puesta en producción).
- **Principio Rector de la Cátedra**: ***"La calidad no se inyecta ni se compra; debe estar embebida en el proceso"***.
- **Axioma Económico**: ***"Trabajar con calidad es infinitamente más barato que hacerlo sin calidad"***.

---

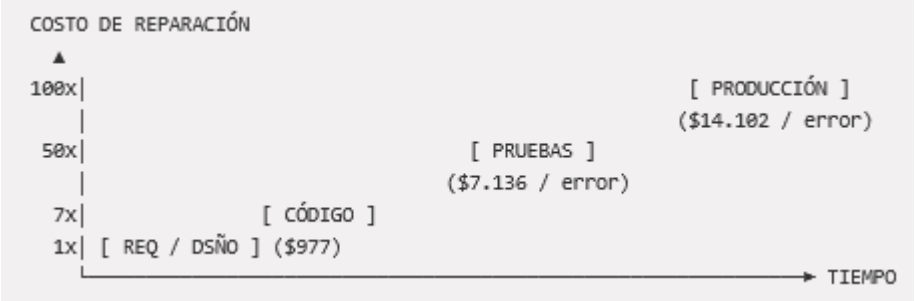
2. La Economía de la Calidad: Regla del Costo Exponencial de Reparación

La justificación financiera de trabajar con calidad radica en cómo se comporta el costo de reparar un defecto a lo largo del tiempo.

Curva del Costo Relativo de Reparación de Defectos (Boehm / Gilb & Graham):

Un error introducido en las etapas iniciales (como en los requerimientos o el diseño) multiplica su costo de corrección a medida que avanza por el ciclo de vida:

Etapa de Detección del Defecto	Costo Relativo Aprox. (USD)	Factor de Multiplicación	Impacto Operativo
Requerimientos / Análisis	~$139	**1x** (Base)	Modificar una especificación o texto de Historia de Usuario.
Diseño Arquitectónico	~$455	**3x - 5x**	Corregir un diagrama o contrato de interfaz de componente.
Codificación (Desarrollo)	~$977	**7x**	El desarrollador corrige el bug en su propia máquina.
Pruebas de Sistema / Integración	~$7.136	**50x**	Falla detectada por testing. Implica retrabajo, re-compilación y re-ejecución de pruebas.
Mantenimiento / Producción	~$14.102+	**100x o más**	Parche de emergencia, afectación a usuarios reales, pérdida de datos o catástrofes operativas.



Los 4 Costos de la Calidad (Cost of Quality - CoQ):

La gestión de proyectos clasifica los costos financieros de la calidad en cuatro categorías:

1. **Costos de Prevención**: Inversión para evitar que ocurran errores (capacitación del personal, establecimiento de estándares, herramientas de integración continua, diseño robusto). *Es la inversión más rentable*.
2. **Costos de Evaluación / Valoración**: Gastos asociados a la inspección y verificación para medir la calidad (ejecución de pruebas automatizadas y manuales, revisiones técnicas, auditorías).
3. **Costos de Falla Interna**: Gastos generados por defectos detectados *antes* de entregar el producto al cliente (retrabajo de código, re-testing, descarte de componentes).
4. **Costos de Falla Externa**: Gastos y pérdidas sufridas cuando el defecto llega al usuario final en producción (soporte técnico de crisis, penalidades contractuales, demandas legales, pérdida de clientes y daño reputacional).

---

3. El Dilema de la Calidad (Bertrand Meyer) y el Software "Suficientemente Bueno"

En el ámbito comercial surge una tensión constante entre el perfeccionismo técnico y las presiones del mercado:

El Dilema (Meyer):

- Si una empresa produce software de **mala calidad**, fracasa porque los clientes rechazan el producto.
- Si una empresa invierte **tiempo y recursos infinitos** buscando un software perfecto, fracasa por agotar sus fondos o perder la ventana comercial de mercado (*time-to-market*).

Software "Suficientemente Bueno" (*Good Enough Software*):

Es la estrategia pragmática de entregar un producto con un nivel de calidad aceptable que añade valor al cliente en el tiempo adecuado, asumiendo la existencia de errores menores no críticos que se corregirán en versiones posteriores (v1.0 → v2.0).

- **Límite y Advertencia de Cátedra (Foco de Examen)**: El enfoque de software "suficientemente bueno" es viable únicamente en software comercial de consumo masivo o WebApps. Es **totalmente inaceptable e irresponsable en software crítico** (médico, automotriz, aeroespacial, sistemas bancarios o de control de tráfico aéreo), donde una falla puede costar vidas humanas o pérdidas financieras multimillonarias.

---

4. Calidad Interna vs. Calidad Externa (Visión Ágil - Kniberg / SCM)

Al desarrollar productos de software, es vital diferenciar dos perspectivas de calidad:

- **Calidad Externa**: Es la calidad percibida directamente por el cliente y los usuarios finales (interfaz de usuario, tiempo de respuesta, funcionalidades visibles). Es **negociable a nivel de alcance** (p. ej., se puede acordar con el Product Owner postergar una funcionalidad para un release posterior para salir antes al mercado).
- **Calidad Interna**: Son las características técnicas invisibles para el usuario pero indispensables para la mantenibilidad y evolución del sistema (arquitectura limpia, legibilidad del código, cobertura de pruebas, ausencia de código duplicado).
- **Regla de Cátedra**: **LA CALIDAD INTERNA NO ES NEGOCIABLE. NUNCA**. Sacrificar la calidad interna introduciendo "apaños rápidos" (*hacks*) genera **deuda técnica**: ahorra minutos hoy, pero paraliza y degrada el desarrollo a mediano y largo plazo.

---

5. Ventajas y Desventajas de Trabajar CON Calidad

Ventajas:

1. **Reducción drástica del retrabajo**: Al prevenir errores en etapas tempranas, los desarrolladores pasan más tiempo construyendo nuevas funcionalidades y menos tiempo "apagando incendios" o depurando bugs en producción.
2. **Mayor previsibilidad y menores costos totales**: Se cumple con la triple restricción del proyecto (tiempo, costo, alcance), evitando desbordes presupuestarios.
3. **Ventaja competitiva y retención de clientes**: Las empresas que trabajan con calidad demostrada (certificando modelos de proceso) sobreviven en el mercado global e incrementan sus beneficios al satisfacer las expectativas del cliente.
4. **Sostenibilidad del equipo (Ritmo Sostenible)**: Reduce el estrés, el trabajo fuera de hora y el desgaste profesional (*burnout*), incrementando la motivación del personal.

Desventajas / Desafíos Iniciales (Percepción de Costo):

1. **Inversión inicial en infraestructura y capacitación**: Exige destinar presupuesto inicial para capacitar al personal, adquirir herramientas e implementar entornos de CI/CD.
2. **Sensación inicial de desaceleración**: La introducción de revisiones técnicas e inspecciones formales parece ralentizar la velocidad de desarrollo al comienzo del proyecto mientras el equipo adquiere la disciplina.
3. **Resistencia cultural**: Requiere vencer la inercia del equipo y la gerencia para transformar hábitos de trabajo arraigados, lo que exige un liderazgo firme y patrocinio gerencial.

---

6. Cuadro Comparativo: Trabajar "PARA" la Calidad vs. Trabajar "CON" Calidad

Criterio / Dimensión	Trabajar "PARA" la Calidad (Reactivo)	Trabajar "CON" Calidad (Preventivo)
Momento de intervención	Al final del ciclo de desarrollo (fase de testing masivo).	Desde el inicio y a lo largo de todo el ciclo de vida.
Objetivo central	Detectar y parchear los errores ya cometidos.	Prevenir la introducción de defectos en el producto.
Ubicación de la responsabilidad	Delegada casi exclusivamente en el equipo de Testing / QA.	**Responsabilidad compartida de todo el equipo** de proyecto.
Costo económico total	**Muy elevado**: Alimentado por el retrabajo masivo y parches en producción.	**Mínimo y optimizado**: Los errores se resuelven cuando son económicos de reparar.
Efecto en el código	Acumulación de deuda técnica y degradación de la arquitectura.	Alta mantenibilidad, código limpio y evolutivo.
Impacto en el equipo	Estrés, horas extras, crisis continuas y alta rotación.	Ritmo sostenible de trabajo y alta motivación.

---

7. Ejemplos Prácticos de la Industria (Casos Reales de Examen)

La falta de trabajo con calidad preventivo ha provocado algunos de los desastres informáticos más costosos de la historia:

- **Cohete Ariane 5 (1996 - Pérdida de $500 millones USD)**: Un error no detectado en la conversión de un número de coma flotante de 64 bits a un entero de 16 bits provocó la autodestrucción del cohete. Una revisión técnica adecuada del software reutilizado lo habría evitado.
- **Mars Climate Orbiter (1999 - Pérdida de $655 millones USD)**: La sonda espacial se perdió en Marte porque un equipo de ingenieros utilizó unidades imperiales (libras/pulgadas) mientras que otro utilizó el sistema métrico internacional, fallando los controles de integración entre componentes.
- **Llamado a revisión de Toyota (2010 - Costo de $3.000 millones USD)**: Un fallo en el software que controlaba el sistema anti-bloqueo de frenos e híbrido obligó a retirar millones de vehículos del mercado.

---

8. Relaciones, Mitos y Trampas Frecuentes de Examen

- **Trampa 1: "Aumentar la cantidad de testers y pruebas incrementa la calidad del software"**: **FALSO**. El testing pone en evidencia la presencia de defectos, pero **no agrega calidad por sí mismo**. Si el código fue mal diseñado, ejecutar más pruebas solo confirmará que está mal construido y aumentará la factura de retrabajo.
- **Trampa 2: "La calidad técnica del código es negociable para llegar a la fecha límite"**: **FALSO**. En el desarrollo profesional y en los marcos ágiles, la calidad interna del producto no se negocia. Si el tiempo apremia, lo único que se negocia y reduce es el **alcance** (*scope*) de las funcionalidades a entregar.
- **Trampa 3: "Invertir en prevención de calidad es un gasto innecesario que retrasa el proyecto"**: **FALSO**. Aunque al inicio requiere esfuerzo de organización, el ahorro logrado en el ciclo de mantenimiento y en la reducción de defectos compensa con creces la inversión inicial (retorno de inversión probado de hasta 10 a 1).

Tema 7: Actividades de aseguramiento de la calidad del software

---

1. Definición, Objetivos y Marco Conceptual de SQA / PPQA

El **Aseguramiento de Calidad del Software (Software Quality Assurance - SQA)**, denominado modernamente en modelos de procesos como **Aseguramiento de Calidad de Proceso y de Producto (PPQA - Process and Product Quality Assurance)**, es una disciplina de soporte transversal ("actividad sombrilla") que acompaña todo el ciclo de vida del software.

Definición Formal

SQA es el conjunto planificado y sistemático de actividades y procedimientos necesarios para proveer la confianza adecuada de que el proceso de desarrollo de software cumple con las políticas y estándares organizacionales, y que los productos de software resultantes satisfacen sus requerimientos funcionales y de calidad especificados.

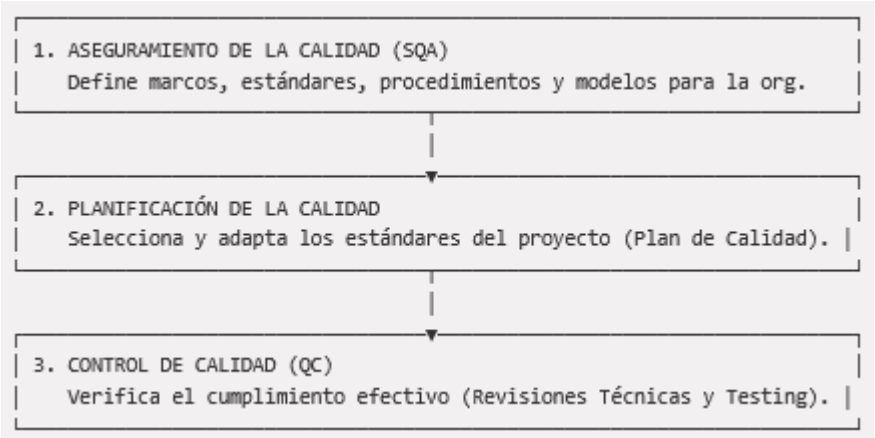
Objetivos Fundamentales de SQA / PPQA:

1. **Evaluar objetivamente** los procesos realizados, los productos de trabajo construidos y los servicios prestados frente a las descripciones de procesos, estándares y procedimientos aplicables.
2. **Asegurar el cumplimiento** de los estándares de proceso y producto adoptados por la organización o exigidos contractualmente por el cliente.
3. **Identificar, documentar y dar seguimiento a las no-conformidades** y desviaciones detectadas en el producto, en el proceso o en los estándares, comunicándolas a la gerencia para su resolución.
4. **Fomentar una "cultura de calidad"** en la organización, promoviendo la idea de que la calidad es una responsabilidad colectiva e indivisible de todo el equipo de desarrollo.

---

2. Las Tres Actividades de la Administración de la Calidad de Software

Según la cátedra y la literatura de referencia (Sommerville/Pressman), la Administración de la Calidad del Software engloba tres actividades interrelacionadas:



1. **Aseguramiento de Calidad (*Quality Assurance*)**:
    - **Función**: Define y establece el marco de procesos, estándares, procedimientos, guías y herramientas organizacionales sobre los cuales se realizarán las comparaciones y evaluaciones.
2. **Planificación de Calidad (*Quality Planning*)**:
    - **Función**: Desarrolla el **Plan de Calidad del Proyecto** (*PACS - Plan de Aseguramiento de Calidad de Software*). Selecciona de la base organizativa cuáles procedimientos y estándares se aplicarán al proyecto específico, adaptándolos a sus características y restricciones. Define también las metas de calidad y cómo se evaluarán.
3. **Control de Calidad (*Quality Control*)**:
    - **Función**: Ejecuta las revisiones e inspecciones operativas para asegurar que el equipo de desarrollo respete efectivamente los procedimientos y estándares establecidos. Existen dos enfoques principales: **Revisiones de Calidad** (inspecciones, evaluaciones de progreso) y **Evaluaciones Automáticas / Mediciones** (métricas de software y testing).

---

3. Funciones y Tareas Específicas del Grupo de Aseguramiento de Calidad (GAC / SQA Team)

El grupo de SQA actúa internamente como el "representante del cliente" en la organización, evaluando el proyecto con perspectiva objetiva e independiente.

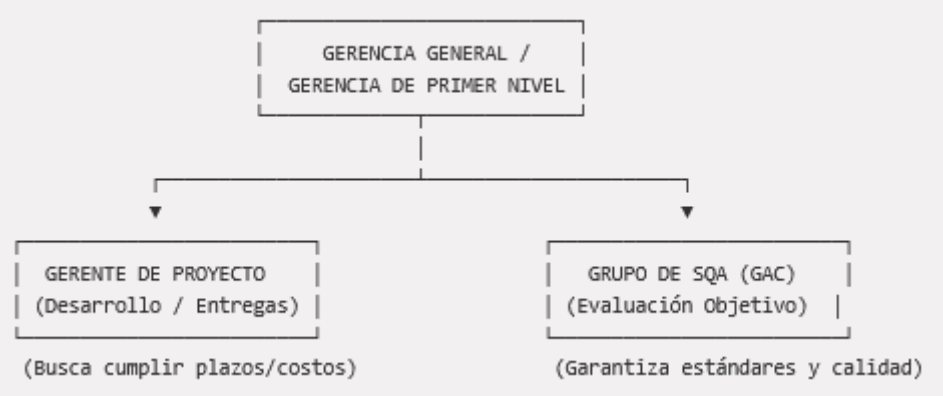
Las 8 Tareas/Funciones Específicas de SQA (Cátedra UTN / SEI):

1. **Desarrollo y adaptación de prácticas de SQA**: Crear herramientas, listas de chequeo (*checklists*), métodos y estándares aplicables a las revisiones de calidad.
2. **Evaluación de la Planificación del Proyecto**: Verificar que el Plan de Proyecto cumpla con las políticas y estándares internos de estimación y seguimiento.
3. **Evaluación del Proceso de Requerimientos**: Garantizar que los requerimientos especificados sean correctos, completos, consistentes y testeables.
4. **Evaluación del Proceso de Diseño**: Revisar que la arquitectura y los modelos de diseño sigan los patrones acordados e implementen adecuadamente las especificaciones.
5. **Evaluación de las Prácticas de Programación**: Verificar el cumplimiento de los estándares de codificación (*coding standards*), legibilidad y buenas prácticas técnicas.
6. **Evaluación de la Integración y Pruebas**: Asegurar que las actividades de testing (unitarias, integración, sistema) se hayan planificado, diseñado y ejecutado correctamente.
7. **Evaluación de la Gestión de Configuración (SCM)**: Controlar que los cambios sobre las líneas base (*baselines*) del producto se realicen bajo procedimientos formales de control de cambios y trazabilidad.
8. **Documentación e Informe de No-Conformidades**: Registrar toda desviación identificada y reportarla a la alta gerencia para coordinar acciones correctivas.

---

4. Estructura Organizacional e Independencia del Grupo de SQA (GAC)

Un aspecto crítico y evaluado con frecuencia en los parciales de la UTN es la **ubicación jerárquica y dependencia orgánica del Grupo de Aseguramiento de Calidad (GAC / SQA Team)**:



Reglas Estrictas de Independencia Orgánica (Foco de Examen):

1. **No debe reportar al Gerente de Proyectos**: Si el área de SQA dependiera del Gerente de Proyecto, existiría un conflicto de intereses insalvable: ante un atraso en el cronograma, el Gerente de Proyecto se vería tentado a "saltear" controles de calidad o ignorar no-conformidades para llegar a la fecha límite.
2. **Dependencia de la Gerencia de Primer Nivel / Gerencia General**: El GAC debe reportar directamente a una autoridad superior que tenga interés genuino en la calidad global de la organización y poder de decisión estratégico.
3. **Máximo de una posición intermedia**: No debe haber más de un nivel jerárquico entre la alta dirección organizativa y el responsable del área de SQA para garantizar la libertad, transparencia y rapidez de reporte.

---

5. El Plan de Aseguramiento de Calidad de Software (PACS / SQA Plan)

El **PACS** es el documento maestro de calidad a nivel de proyecto. Según el estándar **IEEE 730**, el PACS debe definir de forma explícita:

- **Propósito y Alcance**: Identificación del sistema y sus objetivos de calidad.
- **Productos de Trabajo Supervisados**: Listado de todos los modelos, documentos, componentes y código fuente que estarán bajo el alcance de SQA.
- **Estándares y Prácticas Aplicables**: Normas de producto (estándares de código) y de proceso (procedimientos de revisión y SCM).
- **Revisiones y Auditorías**: Calendario, responsabilidades y criterios para ejecutar auditorías de proyecto y revisiones técnicas.
- **Procedimientos de Seguimiento de Errores**: Flujo formal para registrar, clasificar y corregir no-conformidades.
- **Herramientas, Métodos y Métricas**: Recursos técnicos que darán soporte a la recolección de datos de calidad.

---

6. SQA en Enfoques Tradicionales (Definidos) vs. Empíricos (Ágiles)

La aplicación de las actividades de SQA varía según el ciclo de vida del proyecto:

Criterio / Dimensión	Enfoque Tradicional / Definido	Enfoque Empírico / Ágil (Scrum, Kanban)
Mecanismo de Evaluación	**Auditorías formales de proceso y producto** realizadas por auditores/GAC externos al equipo de desarrollo.	**Inspección y adaptación continua** incorporada en la dinámica diaria del equipo autoorganizado.
Rol de SQA / QA	Área independiente que actúa fiscalizando el cumplimiento de fases y documentos.	Miembros de QA integrados al equipo multifuncional (*cross-functional team*); colaboran definiendo pruebas de aceptación desde el Sprint Planning.
Garantía de Calidad de Producto	Revisiones formales de fase (hitos) y fases tardías de pruebas de sistema.	Prácticas ingenieriles continuas: **Definition of Done (DoD)** estricta, **TDD**, **CI/CD** y Revisiones de Pares (*Code Reviews*).
Postura frente a Auditorías Externas	Fomentadas para certificar niveles en modelos como CMMI o ISO.	Cierta resistencia a auditorías externas burocráticas; priorizan la retroalimentación rápida en la **Retrospectiva del Sprint**.

---

7. Relaciones, Mitos y Preguntas Frecuentes de Examen

- **Trampa 1: "SQA es lo mismo que Testing"**: **FALSO**. Testing es una actividad de control de calidad sobre el **producto** cuando este ya existe. SQA es una disciplina integral que abarca el **proceso** a lo largo de todo el ciclo de vida de forma preventiva.
- **Trampa 2: "¿A qué disciplina corresponde la Administración de Configuraciones (SCM)?"**: **Respuesta de Examen**: SCM es una **disciplina de soporte transversal** indispensable que provee la base (control de versiones, trazabilidad, líneas base) para que SQA pueda auditar y asegurar la calidad del producto y del proceso.
- **Trampa 3: "En proyectos ágiles no se realizan actividades de SQA"**: **FALSO** (pregunta típica de examen). En ágil las actividades de SQA no desaparecen; se transforman de auditorías burocráticas a **prácticas automatizadas y continuas** (TDD, integración continua, revisiones entre pares, cumplimiento estricto de la DoD).

Tema 8: Modelos de calidad: CMMI, SPICE e ISO

---

1. Definición, Objetivos y Utilidad de los Modelos de Calidad

Los **modelos de calidad de software** son marcos de referencia (*benchmarks*) teóricos y estructurados que establecen un conjunto de mejores prácticas, guías, requerimientos de proceso o atributos de producto.

- **Objetivo principal**: Evaluar la capacidad actual de los procesos de una organización o las características de un producto, acreditar/determinar su nivel de madurez e identificar oportunidades para la **mejora continua de procesos (Software Process Improvement - SPI)**.
- **Aclaración clave de examen**: Un modelo de calidad **no es un proceso de software en sí mismo** ni una metodología de desarrollo específica. Es una guía contra la cual se compara (*benchmark*) el proceso propio que ha sido definido e instanciado en la organización.

---

2. Capability Maturity Model Integration (CMMI)

CMMI es el marco conceptual de mejora y evaluación de procesos para el desarrollo, mantenimiento y operación de software y sistemas más difundido globalmente.

- **Origen e Institución**: Fue desarrollado por el **Software Engineering Institute (SEI)** de la Universidad Carnegie Mellon, financiado inicialmente por el Departamento de Defensa de los EE. UU. (DoD) como evolución del antiguo SW-CMM. Posteriormente, el CMMI Institute fue adquirido por **ISACA**.
- **Evaluación y Acreditación (Pregunta Frecuente)**: CMMI **NO es una norma y NO se "certifica"**. Las organizaciones se evalúan o **acreditan** en un nivel de madurez o capacidad a través de un método de evaluación oficial denominado **SCAMPI** (*Standard CMMI Appraisal Method for Process Improvement*) conducido por profesionales autorizados llamados **Lead Appraisers**.

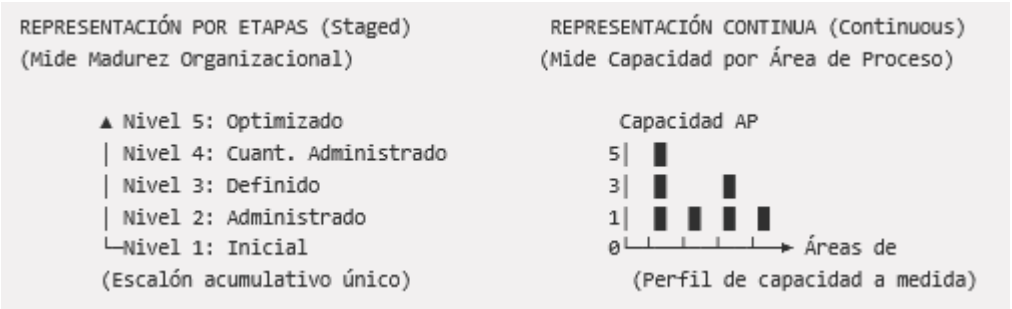
A. Las 3 Constelaciones de CMMI

CMMI agrupa sus guías según el modelo de negocio de la organización en tres conjuntos estructurados o "constelaciones":

1. **CMMI-DEV (*Development*)**: Guía para medir, monitorear y administrar los procesos de desarrollo y mantenimiento de software y sistemas.
2. **CMMI-SVC (*Services*)**: Guía para la prestación e integración de servicios internos o externos a clientes.
3. **CMMI-ACQ (*Acquisition*)**: Guía para la selección, gestión y adquisición de productos y servicios provistos por terceros.
    - *Nota*: Existen 16 Áreas de Proceso (PAs) fundacionales que son comunes a las tres constelaciones.

B. Las Dos Representaciones de CMMI-DEV

CMMI-DEV ofrece dos formas estructurales distintas de organizar y evaluar las Áreas de Proceso:



1. **Representación Por Etapas (*Staged*)**:
    - **Foco**: Evalúa la **Madurez Organizacional** de la empresa como un todo.
    - **Estructura**: Define **5 Niveles de Madurez** (del 1 al 5) acumulativos:
        - *Nivel 1 - Inicial*: Procesos impredecibles, inmaduros y reactivos. El éxito depende del esfuerzo heroico e individual.
        - *Nivel 2 - Administrado*: Los proyectos garantizan que los requerimientos se gestionan y que los procesos se planifican, miden, monitorean y controlan.

        - *Nivel 3 - Definido*: Los procesos de ingeniería y gestión están estandarizados, documentados e integrados a nivel organizacional.
        - *Nivel 4 - Cuantitativamente Administrado*: Se aplican técnicas estadísticas y métricas cuantitativas para controlar el rendimiento de los subprocesos.
        - *Nivel 5 - Optimizado*: Foco en la mejora continua de procesos a través de la innovación tecnológica y la resolución de causas raíz de fallas.
    - **Regla de Acreditación**: Es **acumulativa**. Para alcanzar el Nivel 3, se deben cumplir obligatoriamente todas las Áreas de Proceso asociadas al Nivel 2 y al Nivel 3.
2. **Representación Continua (*Continuous*)**:
    - **Foco**: Evalúa la **Capacidad de un Área de Proceso (AP)** específica de forma individual.
    - **Estructura**: Define **6 Niveles de Capacidad** (del 0 al 5) para cada AP:
        - *Nivel 0 (Incompleto)* / *Nivel 1 (Realizado)* / *Nivel 2 (Gestionado)* / *Nivel 3 (Definido)* / *Nivel 4 (Cuantitativamente Administrado)* / *Nivel 5 (Optimizado)*.
    - **Ventaja**: Permite a la organización seleccionar el orden de las mejoras atendiendo a sus objetivos de negocio prioritarios (p. ej. profundizar SCM sin necesidad de certificar toda la organización).

C. Áreas de Proceso (PAs) Clave en CMMI v1.3 (Foco de Examen):

- **Nivel 2 (Administrado)**: *Gestión de Requerimientos (REQM), Planificación de Proyectos (PP), Monitoreo y Control (PMC), Gestión de Acuerdos con Proveedores (SAM), Medición y Análisis (MA), Aseguramiento de Calidad de Proceso y Producto (PPQA), Gestión de Configuración (CM)*.
- **Nivel 3 (Definido)**: *Desarrollo de Requerimientos (RD), Solución Técnica (TS), Integración de Producto (PI), Verificación (VER), Validación (VAL), Gestión de Riesgos (RSKM), Definición del Proceso Organizacional (OPD)*.

---

3. Modelo SPICE (ISO/IEC 15504) y el Marco IDEAL

A. Modelo SPICE (Software Process Improvement Capability Determination)

Es el estándar internacional promovido por la ISO/IEC para la evaluación objetiva de la capacidad de los procesos de desarrollo de software.

- **Estructura**: Define **6 Niveles de Madurez/Capacidad** para evaluar las organizaciones:
    - *Nivel 0: Organización inmadura* (no ejecuta efectivamente los procesos).
    - *Nivel 1: Organización básica* (alcanza los objetivos de los procesos).
    - *Nivel 2: Organización gestionada* (los productos de trabajo se controlan y mantienen).
    - *Nivel 3: Organización establecida* (utiliza procesos adaptados basados en estándares).
    - *Nivel 4: Organización predecible* (gestiona cuantitativamente los procesos).
    - *Nivel 5: Organización optimizando* (mejora continua orientada a metas de negocio).

B. Modelo IDEAL (SEI / ISO)

Es un ciclo metodológico de 5 fases diseñado para estructurar y guiar la **ejecución de proyectos de mejora de procesos** dentro de una organización:

1. **Initiating (Inicialización)**: Lograr el patrocinio ejecutivo, definir los objetivos de negocio y establecer la infraestructura del proyecto de mejora.
2. **Diagnosing (Diagnóstico)**: Evaluar el estado actual de los procesos (*as-is*) mediante SPICE o CMMI para identificar brechas.
3. **Establishing (Establecimiento)**: Priorizar oportunidades de mejora, elaborar el plan de acción y diseñar las soluciones técnicas/estándares.
4. **Acting (Acción / Ejecución)**: Implementar las mejoras mediante **proyectos piloto** para validar el impacto antes de generalizarlas.
5. **Learning (Aprendizaje)**: Analizar los resultados del piloto, consolidar lecciones aprendidas y desplegar la solución a toda la organización.

---

4. Normas ISO en la Gestión de Calidad de Software

- **ISO 9001 / TickIT**: Norma genérica de sistemas de gestión de la calidad. Evalúa la consistencia operacional del proceso. *A diferencia de CMMI, la ISO 9001 se certifica formalmente*.
- **ISO/IEC 12207**: Estándar que establece el marco de referencia unificado para los procesos del ciclo de vida del software (procesos primarios, de soporte y organizacionales).
- **ISO/IEC 25010 (SQuaRE)**: Reemplaza a la norma ISO/IEC 9126 como el modelo internacional para la **evaluación de la calidad de producto de software** (evalúa 8 atributos no funcionales: *Adecuación Funcional, Eficiencia de Desempeño, Compatibilidad, Usabilidad, Fiabilidad, Seguridad, Mantenibilidad, Portabilidad*).

---

5. Tablas Comparativas de Modelos

A. CMMI Por Etapas vs. CMMI Continuo

Criterio / Aspecto	CMMI Por Etapas (*Staged*)	CMMI Continuo (*Continuous*)
Objeto de Evaluación	La **Madurez Organizacional** de la empresa como un todo.	La **Capacidad de un Área de Proceso** individual.
Estructura de Niveles	5 Niveles de Madurez (1 a 5).	6 Niveles de Capacidad (0 a 5) por cada Área de Proceso.
Flexibilidad de Mejora	**Rígida**: Sigue una ruta predefinida y acumulativa por etapas.	**Flexible**: La organización elige libremente qué áreas de proceso desea priorizar.
Resultado de Evaluación	Un único número o nivel global para la organización.	Un **perfil de capacidad** con mediciones individuales por cada AP.

B. Comparación Global: CMMI vs. SPICE vs. ISO 9001

Dimensión	CMMI (DEV v1.3 / v2.0)	SPICE (ISO/IEC 15504)	ISO 9001 (TickIT)
Mecanismo de Evaluación	Evaluado / Acreditado vía Lead Appraisers (SCAMPI).	Evaluado por asesores mediante modelo de capacidad.	**Certificado formalmente** por organismos internacionales.
Enfoque Principal	Proceso de ingeniería, gestión y soporte de software/sistemas.	Evaluación y mejora de procesos de software.	Sistema de gestión de la calidad operacional a nivel empresa.
Foco Geográfico / Adopción	Masivo en EE. UU., Latinoamérica y contratistas internacionales.	Ampliamente adoptado en la Unión Europea.	Estándar global multisectorial.

---

6. CMMI frente a los Métodos Ágiles y Mitos de Examen

Un debate común en la cátedra es la compatibilidad entre modelos de madurez y agilidad:

- **Diferencias Filosóficas**: CMMI persigue la **predictibilidad, uniformidad organizacional y reducción de variabilidad**; los métodos ágiles priorizan la **adaptabilidad, innovación del proyecto y respuesta rápida al cambio**.
- **Compatibilidad Real**: **Son totalmente compatibles y complementarios**. CMMI indica *qué* resultados u objetivos deben alcanzarse (*what*), mientras que las metodologías ágiles (como Scrum o XP) definen *cómo* ejecutar el trabajo diario (*how*). Existen numerosos casos documentados de organizaciones ejecutando **Scrum en Nivel 5 de CMMI**.
- **Mitos de Examen**:
    1. *¿CMMI se certifica?* **FALSO**: CMMI se **evalúa o acredita** por un Lead Appraiser a través de SCAMPI.
    2. *¿CMMI exige una cascada tradicional?* **FALSO**: CMMI es agnóstico del ciclo de vida; se puede implementar con iteraciones cortas y prácticas ágiles.

Tema 9: Auditorías al software: auditorías de proyecto y al grupo de calidad

---

1. Definición Formal, Propósito y Justificación

Definición Formal de Auditoría de Software

Según el estándar **IEEE Std 1028** y la cátedra de la UTN, una auditoría de calidad de software es una **evaluación independiente e imparcial** de los productos o procesos de software para asegurar el cumplimiento con estándares, lineamientos, especificaciones, procedimientos y planes contractuales u organizacionales. Se fundamenta en criterios objetivos y en la recolección de evidencia documentada sobre:

1. La forma o contenido de los productos a ser desarrollados (artefactos).
2. El proceso por el cual los productos son efectivamente desarrollados.
3. La forma en que debe medirse el cumplimiento con los estándares o lineamientos establecidos.

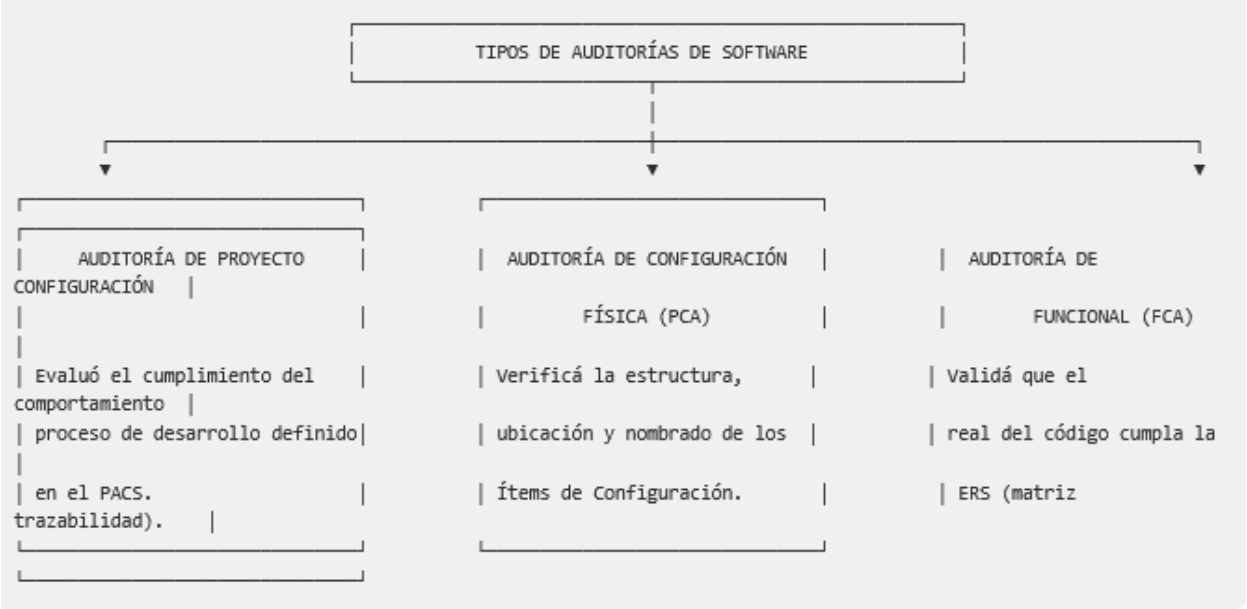
Axiomas y Razones de Ser (*¿Por qué auditar?*):

- **Independencia y Objetividad**: Permite obtener una opinión imparcial, libre de los condicionamientos y presiones operativas del equipo de desarrollo (como el cumplimiento de fechas límite).
- **Garantía al Cliente**: Asegura al cliente que el software se está construyendo según las pautas de calidad convenidas.
- **Visibilidad Gerencial**: Otorga visibilidad a la alta dirección sobre el estado real de los proyectos y los procesos de trabajo.
- **Identificación de Oportunidades de Mejora**: Revela fortalezas y áreas de insatisfacción potencial antes de que afecten al usuario final.

---

2. Tipos Principales de Auditorías de Software

Dentro de la disciplina de Aseguramiento de Calidad (PPQA / SQA) y la Administración de la Configuración (SCM), la cátedra distingue tres tipos fundamentales de auditorías:



A. Auditoría de Proyecto

- **Foco**: Se concentra en evaluar si el proyecto se ha ejecutado respetando el **proceso de desarrollo que se definió y adaptó** en el **Plan de Aseguramiento de Calidad de Software (PACS)**.
- **Mecánica de Trabajo**: El auditor toma evidencias objetivas de la ejecución diaria del equipo (actas de reunión, revisiones de diseño, registros de pruebas, estados de tareas) y las contrasta con los procedimientos documentados en la Especificación de Requerimientos de Software (ERS) y la Documentación del Diseño de Software (DDS).
- **Objetivo de Consistencia**: Verificar objetivamente que:
    - Las interfaces de hardware y software sean consistentes con los requerimientos.
    - El diseño del producto satisfaga los requerimientos funcionales.
    - El código fuente construido sea consistente con la arquitectura y el diseño.

B. Auditoría de Configuración Física (PCA - Physical Configuration Audit)

- **Foco**: Compara el código fuente e Ítems de Configuración (ICS) generados contra la documentación técnica y de soporte.
- **Aspecto Técnico**: Verifica la estructura y la integridad de la línea base congelada: *¿Están todos los archivos entregables en el repositorio indicado? ¿Respetan el esquema de nombrado y versionado definido en el plan de SCM?*.
- **Efecto de Control**: Es una actividad de **Verificación**. Su propósito es asegurar que la documentación entregada sea consistente y describa correctamente al código desarrollado.

C. Auditoría de Configuración Funcional (FCA - Functional Configuration Audit)

- **Foco**: Compara el software ejecutablemente construido con la **Especificación de Requerimientos de Software (ERS)**.
- **Aspecto Técnico**: Asegura que el código implemente **única y completamente** la funcionalidad y métricas de desempeño requeridas por el cliente. El auditor de QA evalúa y exige que la **matriz de rastreabilidad / trazabilidad de requerimientos** se encuentre 100% actualizada.
- **Efecto de Control**: Es una actividad de **Validación**.

Regla Crítica de Dependencia (Pregunta Típica de Parcial):

> ***"La Auditoría de Configuración Física (PCA) se realiza siempre antes que la Auditoría de Configuración Funcional (FCA). Si la Auditoría Física falla o no se aprueba, la Auditoría Funcional NO se ejecuta"***.
> *Fundamento*: No tiene sentido validar el comportamiento de un código cuyas líneas base, estructura de archivos o documentación están desorganizadas, incompletas o sin control de versión.

---

3. Auditorías al Grupo de Calidad (SQA / PPQA)

Además de auditar los proyectos de desarrollo, los modelos de calidad contemplan las **auditorías externas al propio grupo de aseguramiento de calidad (GAC / SQA Team)**:

- **Concepto**: Son evaluaciones conducidas por personal o auditores externos a la organización (consultores independientes o entes acreditadores) para auditar al equipo de SQA.
- **Propósito**: Determinar si los miembros de QA están realizando sus evaluaciones, revisiones e inspecciones internas de forma correcta, objetiva e imparcial.
- **Garantía Organizacional**: Evita el "cegamiento por rutina" del grupo interno de calidad y evalúa la efectividad real del Plan de Aseguramiento de Calidad de Software (PACS) frente a marcos como CMMI, SPICE e ISO 9001.

---

4. Roles en el Proceso de Auditoría

En una auditoría formal participan tres roles con responsabilidades claramente delimitadas:

Rol	Definición y Responsabilidades Principales
Auditado	Generalmente es el **Líder de Proyecto** o el responsable del área/proceso evaluado. Participa proponiendo o acordando la fecha, entrega la evidencia objetiva requerida, responde las consultas del auditor y es el **responsable de formular y ejecutar el plan de acciones correctivas** ante las no-conformidades encontradas.
Auditor / Equipo Auditor	Persona o grupo **independiente del proyecto y del área auditada**. Acuerda la fecha, comunica el alcance, recolecta y analiza la evidencia objetiva relevante, redacta el **reporte de auditoría** y realiza el seguimiento del cierre de los planes de acción.
Gerente de SQA / Alta Gerencia	Responsable orientado a la **gestión global del programa de auditorías**. Prepara el plan anual de auditorías, calcula presupuestos, asigna auditores y actúa como árbitro resolviendo disputas sobre no-conformidades entre auditor y auditado.

---

5. Auditorías en Enfoques Tradicionales (Definidos) vs. Ágiles (Empíricos)

El concepto de auditoría sufre una profunda transformación operacional al cambiar de paradigma:

Dimensión	Enfoque Tradicional / Definido	Enfoque Ágil / Empírico (Scrum, Kanban)
Filosofía de Control	Basado en el control externo, inspección de líneas base congeladas y cumplimiento de documentación.	Filosofía "sin fricción": SCM y SQA están al servicio del equipo y no para controlar a los desarrolladores.
Ejecución de Auditorías de Proyecto	Realizadas por **auditores independientes y externos al proyecto** o a la empresa.	**Sustituidas internamente**: El propio equipo autoorganizado realiza la inspección y adaptación de su proceso en la **Sprint Retrospective** y del producto en la **Sprint Review**.
Auditorías de Configuración (PCA/FCA)	Revisiones formales ante hitos o congelamiento de versión.	**Reemplazadas por automatización**: Integración Continua (CI), servidores de build y suites de pruebas automatizadas que validan cada commit.
Postura del Equipo	Aceptadas como requisito formal de contrato o certificación.	Fuerte resistencia a auditorías externas formales por considerarlas burocráticas y generadoras de desperdicio.

---

6. Métricas de Auditoría

Para evaluar la eficiencia del propio proceso de auditoría, se recopilan métricas específicas:

- **Esfuerzo por auditoría**: Cantidad de horas-hombre consumidas en preparar y ejecutar la auditoría.
- **Cantidad de desviaciones / No-Conformidades**: Total de observaciones encontradas, clasificadas por severidad o por Área de Proceso (PA) de CMMI.
- **Duración de la auditoría**: Tiempo de calendario desde el inicio de la revisión hasta la emisión del informe.
- **Tasa de resolución de acciones correctivas**: Porcentaje de no-conformidades cerradas dentro del plazo estipulado.

---

7. Relaciones, Mitos y Trampas Frecuentes de Examen

- **Trampa 1: Confundir Auditorías con Revisiones Técnicas (*Peer Reviews*)**:
    - Las **Revisiones Técnicas** son realizadas por **colegas / pares** del equipo de desarrollo para detectar errores en el código o diseño antes de avanzar.
    - Las **Auditorías** son realizadas por **personal independiente y externo al equipo de desarrollo** para evaluar el cumplimiento de procesos y estándares.
- **Trampa 2: "En metodologías ágiles SCM no realiza auditorías de ningún tipo"**: **Respuesta de Examen**: En ágil, las auditorías formales externas se eliminan o reducen al mínimo, siendo la **única actividad de la disciplina SCM que la filosofía ágil no soporta ni promueve activamente**. En su lugar, el control de configuración y calidad se delega a herramientas de Integración Continua (CI) y a la inspección interna del equipo.
- **Trampa 3: "La auditoría objetiva la puede hacer el mismo Líder de Proyecto"**: **FALSO**. Debe existir independencia de criterio (sin condicionamientos sobre lo auditado) e independencia jerárquica/salarial del elemento auditado.

Tema 10: Proceso de auditorías

---

1. Definición, Objetivos y Principios del Proceso de Auditoría

El **proceso de auditoría de software** es la secuencia estructurada, formal y planificada de actividades mediante la cual un auditor (o equipo auditor) independiente recolecta y analiza evidencia objetiva para determinar el grado de cumplimiento de un proyecto o producto de software respecto a los procesos, estándares, procedimientos y requerimientos formalmente documentados.

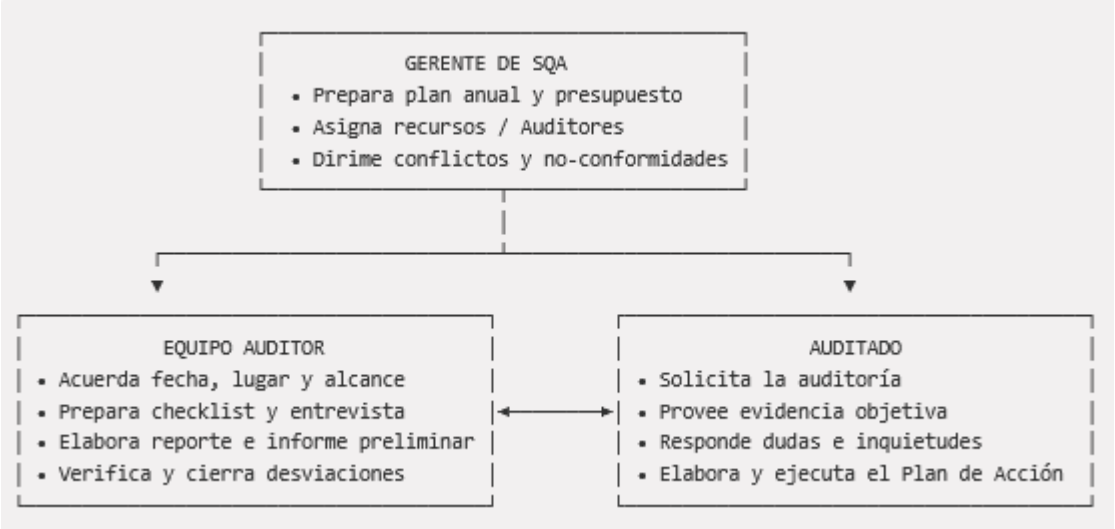
Principios Fundamentales del Proceso (Cátedra UTN):

- **Transparencia ("No son sorpresivas")**: Una auditoría de software nunca se realiza de forma intempestiva o secreta. Es un proceso acordado previamente entre el auditor y el auditado, fijando fecha, lugar, objetivo y alcance.
- **Basado en Evidencia Objetiva**: El auditor no juzga por impresiones subjetivas ni por lo que "se dice que se hace", sino constatando registros, documentos, configuraciones y artefactos reales (*"El auditor escucha lo que dicen que hacen y comprueba la documentación que muestra lo que debería hacerse"*).
- **Orientado a la Mejora del Sistema, no a Castigar Personas**: El propósito del proceso de auditoría es evaluar el cumplimiento y la efectividad de los procesos de trabajo organizacionales y del proyecto. No se utiliza para evaluar el desempeño personal de los desarrolladores.

---

2. Roles y Responsabilidades en el Proceso de Auditoría

En la ejecución del proceso de auditoría intervienen tres roles principales con responsabilidades operativas y de gestión bien delimitadas:



A. Auditado

- **Quién es**: Por lo general es el **Líder de Proyecto** o el responsable del área / proceso objeto de evaluación.
- **Responsabilidades**:
    1. Solicitar la auditoría o acordar su fecha, lugar y alcance.
    2. Proporcionar la documentación previa y la evidencia objetiva requerida durante la entrevista.
    3. Contestar y pedir clarificaciones sobre el reporte preliminar.
    4. **Formular y enviar el Plan de Acción** para corregir las desviaciones encontradas, indicando las fechas de compromiso de resolución.
    5. Ejecutar las acciones correctivas e informar formalmente su cumplimiento al auditor.

B. Auditor (o Equipo Auditor)

- **Quién es**: Persona o grupo de trabajo **independiente del proyecto y del área auditada** (perteneciente al Grupo de SQA o a un ente externo).
- **Responsabilidades**:
    1. Acordar la fecha y comunicar formalmente el alcance de la auditoría.

2. Recolectar y analizar la evidencia objetiva relevante y suficiente.
3. Conducir las reuniones de apertura, entrevistas y reunión de cierre.
4. Confeccionar el checklist, redactar el informe preliminar y el reporte final de auditoría.
5. Revisar, evaluar y aprobar el Plan de Acción presentado por el auditado.
6. **Realizar el seguimiento, verificar el cumplimiento de las correcciones y cerrar formalmente las desviaciones**.

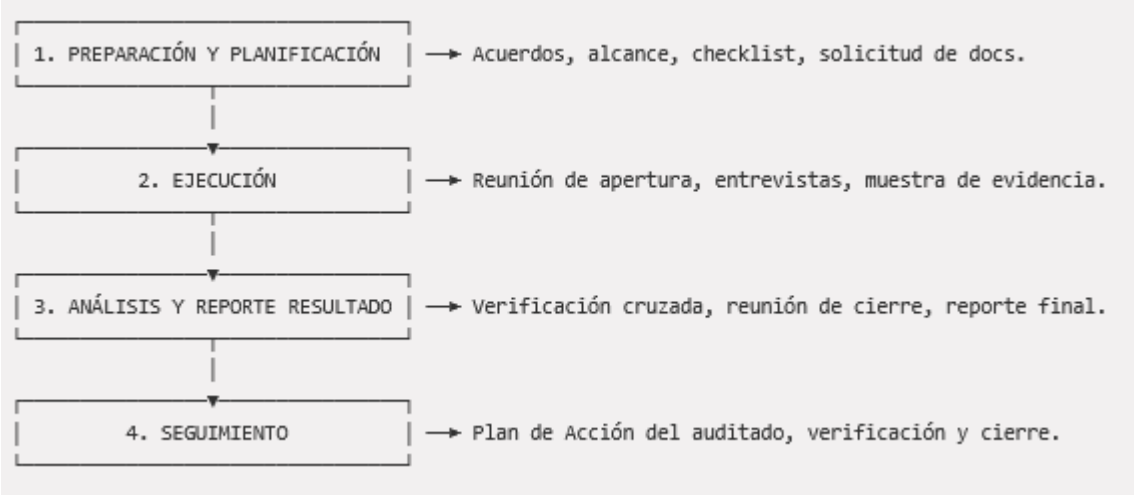
C. Gerente de SQA (Software Quality Assurance)

- **Quién es**: Responsable de la gestión global del programa de aseguramiento de calidad de la empresa.
- **Responsabilidades**:
    1. Preparar el plan anual de auditorías de la organización.
    2. Calcular los costos y asignar los recursos y auditores para cada auditoría.
    3. Actuar como autoridad de resolución cuando existen desacuerdos o no-conformidades no resueltas entre el auditor y el auditado.

---

3. Las 4 Etapas del Proceso de Auditoría (Foco de Examen)

El proceso de auditoría se desarrolla de manera lineal a través de cuatro etapas fundamentales:



Etapa 1: Preparación y Planificación

- **Mecánica**:
    1. El auditado solicita formalmente la auditoría o se activa según el Plan de Aseguramiento de Calidad de Software (PACS).
    2. Auditor y auditado acuerdan el lugar, fecha, hora, objetivo, alcance y la lista de participantes.
    3. El auditor solicita documentación preliminar (p. ej. PACS, ERS, Plan de SCM) para su revisión previa.
    4. El auditado prepara la documentación e información requerida.
    5. **Herramientas de Preparación**: El auditor prepara el **Checklist de Auditoría** (lista de preguntas estandarizadas de mínima) y selecciona las métricas de auditoría a recolectar.

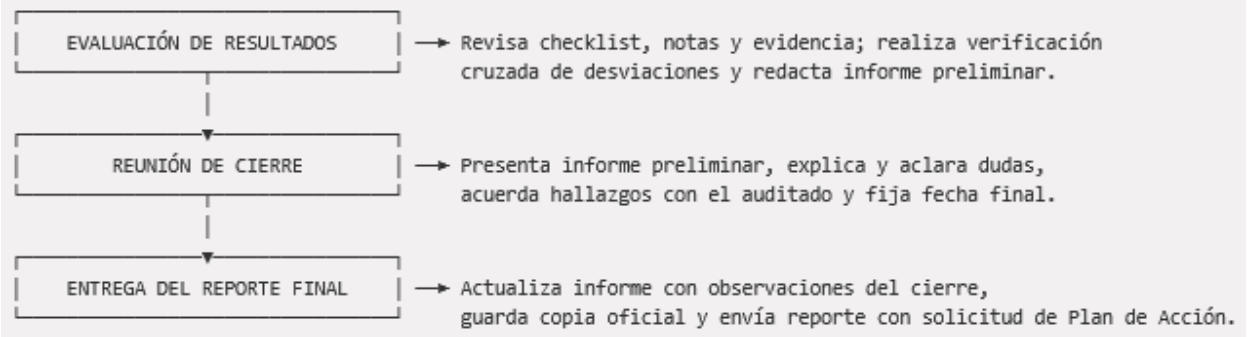
Etapa 2: Ejecución

Comprende el trabajo de campo de la auditoría y se divide en dos fases:

1. **Reunión de Apertura**:
    - Presentación de los participantes y del equipo auditor.
    - Revisión del cronograma, objetivo y alcance acordado.
    - Clarificación de dudas sobre el proceso, explicación de los formalismos, normas de conducta y solicitud de cooperación.
2. **Ejecución Propiamente Dicha (Entrevistas y Recolección)**:
    - El auditor realiza preguntas al auditado y solicita evidencia objetiva para respaldar las respuestas.

- El auditado responde y muestra la evidencia física o digital (registros, código, actas, matrices).
- El auditor completa la lista de chequeo (*checklist*), toma anotaciones y comunica la fecha estimada en la que se entregará el informe preliminar.

Etapa 3: Análisis y Reporte de Resultados



1. **Evaluación de los Resultados**: El auditor analiza las anotaciones, los registros y el checklist completado. Realiza una **verificación cruzada de las desviaciones detectadas contra el proceso definido** y redacta el *informe preliminar*.
2. **Reunión de Cierre**: El auditor presenta el reporte preliminar al auditado. Se explican los resultados, el auditado puede solicitar clarificaciones o fundamentar posturas, y se acuerda la lista definitiva de hallazgos. Se fija la fecha exacta para la entrega del reporte final.
3. **Entrega del Reporte Final**: El auditor actualiza el informe preliminar incorporando los comentarios de la reunión de cierre, redacta el **Reporte Final de Auditoría**, guarda una copia de respaldo y lo envía formalmente al auditado junto con la solicitud del Plan de Acción.

Etapa 4: Seguimiento (*Follow-up*)

Es la fase que garantiza la efectividad del proceso de auditoría:

1. El auditado analiza las desviaciones reportadas y elabora un **Plan de Acción** que especifica las actividades correctivas y la fecha comprometida de realización.
2. El auditor revisa y aprueba el Plan de Acción.
3. El auditado ejecuta las acciones planteadas y notifica formalmente su cumplimiento.
4. El auditor verifica en el terreno/código el cumplimiento real de las correcciones, **procede al cierre de las desviaciones** e informa formalmente el cierre a la gerencia y al auditado.

---

4. Clasificación de Hallazgos y Resultados de Auditoría

Durante la evaluación, el auditor clasifica las evidencias encontradas en tres tipos de hallazgos:

Tipo de Hallazgo	Definición y Características	Exigencia de Plan de Acción
Buenas Prácticas	Práctica, procedimiento o herramienta implementada que ha sido desarrollada de forma **superadora o mucho mejor de lo esperado** en los estándares. Sirven como modelo para extender al resto de la organización.	**No requiere**. Se destaca como fortaleza.
Desviaciones (No-Conformidades)	Cualquier incumplimiento o no-conformidad respecto al proceso definido, requerimientos documentados o pautas de SCM. Se subdividen en: • *No Adecuado*: Incumplimiento grave / falla en el producto o proceso. • *Necesita Mejora*: Se ejecuta pero con deficiencias.	**OBLIGATORIO**. El auditado debe elaborar y ejecutar un Plan de Acción con fecha de cumplimiento.
Observaciones	Opiniones del auditor sobre condiciones o prácticas que, si bien no constituyen una desviación en el presente, **podrían derivar en un riesgo o futura desviación** si no se atienden.	**No requiere Plan de Acción obligatorio**. Queda a consideración del equipo corregirlo.

---

5. Herramientas, Técnicas y Habilidades del Auditor

A. Herramientas y Técnicas Técnicas:

1. **Checklist (Lista de Chequeo)**: Instrumento clave para garantizar que la auditoría sea consistente e independiente de quién la ejecute. Define preguntas de mínima (fecha, proyecto, auditado, auditor, fase y puntos a controlar).
2. **Muestreo**: Selección de una muestra representativa de artefactos, líneas de código o registros cuando no es factible revisar el 100% del proyecto.
3. **Revisión de Registros y Herramientas Automatizadas**: Inspección de bitácoras de CI/CD, repositorios, actas de CCB y métricas.

B. Técnicas de Cuestionario y Habilidades Blandas del Auditor:

- **Secuencia de Preguntas en Entrevista**:
    1. *Preguntas Abiertas* al inicio (quién, cómo, cuándo, qué, dónde) para permitir al auditado explayarse.
    2. *Preguntas Cortas y Puntuales* en la fase media.
    3. *Preguntas Cerradas* al final para clarificar conceptos y confirmar datos específicos.
- **Conducta del Auditor**: Escuchar sin interrumpir, tomar notas minuciosas, observar el lenguaje corporal del auditado, manejar el propio lenguaje corporal y **repetir lo escuchado** para confirmar que se ha comprendido correctamente antes de anotar una posible desviación.

---

6. Métricas del Proceso de Auditoría

Para medir la eficiencia y el desempeño de la propia disciplina de auditorías, las organizaciones recolectan métricas específicas:

- **Esfuerzo por auditoría**: Horas-hombre dedicadas por el auditor y el auditado en las 4 etapas.
- **Cantidad de desviaciones encontradas**: Total de hallazgos clasificados por gravedad o por Área de Proceso (PA) de CMMI.
- **Duración de la auditoría**: Tiempo de calendario (días) transcurrido entre la reunión de apertura y la entrega del reporte final.
- **Tasa de cierre de desviaciones**: Porcentaje de acciones correctivas verificadas y cerradas dentro del tiempo estipulado.

7. Cuadro Comparativo: Proceso de Auditoría vs. Proceso de Inspección (Revisión Técnica)

Dimensión / Criterio	Proceso de Auditoría	Proceso de Inspección (Revisión Técnica)
Propósito Central	Revelar qué se está haciendo en la realidad frente a lo que se comprometió a hacer (procesos/estándares).	Encontrar errores en un artefacto lo antes posible para evitar su propagación.
Roles Participantes	Auditor, Auditado y Gerente de SQA.	Autor, Moderador, Anotador, Lector e Inspectores (colegas/pares).
Objeto de Evaluación	Cumplimiento del proceso de desarrollo, planes de proyecto, SCM y requerimientos.	Un artefacto de trabajo específico (código fuente, documento de diseño, ERS).
Naturaleza del Reporte	Reporte de auditoría con clasificación de Desviaciones, Observaciones y Buenas Prácticas.	Reporte de inspección con lista de errores/defectos a corregir por el autor.
Etapas del Proceso	1. Planificación → 2. Ejecución → 3. Análisis y Reporte → 4. Seguimiento.	1. Planificación → 2. Visión General → 3. Preparación → 4. Reunión → 5. Corrección → 6. Seguimiento.

---

8. Relaciones, Mitos y Preguntas Frecuentes de Examen

- **Mito 1: "El auditor es quien debe proponer la solución y reparar las desviaciones encontradas"**:
  - **FALSO (Pregunta típica de parcial)**: El auditor únicamente identifica y documenta la desviación. Es **responsabilidad exclusiva del auditado** proponer el Plan de Acción con las soluciones técnicas y ejecutarlo.
- **Mito 2: "Las auditorías en Scrum se realizan exactamente igual que en proyectos tradicionales"**:
  - **FALSO**: En entornos ágiles empíricos (como Scrum), las auditorías externas formales sobre el proceso se sustituyen por la **inspección y adaptación continua** que realiza el propio equipo en la ceremonia de **Sprint Retrospective** y sobre el producto en la **Sprint Review**.
- **Mito 3: "Las Observaciones requieren un Plan de Acción obligatorio"**:
  - **FALSO**: Únicamente las **Desviaciones (No-Conformidades)** exigen un Plan de Acción obligatorio con compromiso de fecha de resolución. Las Observaciones son sugerencias preventivas de revisión.

Tema 11: Calidad de producto: planificación, niveles y tipos de pruebas

---

1. Definición, Objetivos y Conceptos Fundamentales del Testing de Software

El **Testing de Software** (Pruebas de Software) es el proceso dinámico de ejecución de un sistema o sus componentes sobre datos artificiales con el objetivo principal de **demostrar que el software cumple con los requerimientos especificados y de descubrir defectos antes de su puesta en producción**.

A. Verificación vs. Validación (El Principio de V&V - Boehm)

Dentro del control de calidad de producto se distingue entre dos conceptos complementarios:

- **Verificación**: *"¿Estamos construyendo el producto correctamente?"*. Evalúa si el software cumple con las especificaciones técnicas y los estándares definidos en las fases previas (orientado al proceso y a la arquitectura interna).
- **Validación**: *"¿Estamos construyendo el producto correcto?"*. Evalúa si el software en ejecución satisface las necesidades reales, expectativas y requerimientos del cliente en su entorno operacional.

B. La Cadena de Error, Defecto y Falla (Foco de Examen)

Es indispensable diferenciar estrictamente la terminología técnica recomendada por la cátedra e ISTQB:

- **Error**: Acción humana equivocada cometida por el desarrollador, analista o diseñador (p. ej., un malentendido de la regla de negocio o una equivocación al escribir un operador lógico).
- **Defecto / Falla Interna (*Bug*)**: Imperfección estática presente en el producto de trabajo (código fuente, documento de diseño o ERS) resultante de un error humano.
- **Falla Externa (*Failure*)**: Manifestación o comportamiento dinámico incorrecto observable en el software durante su ejecución al impactar sobre un defecto presente en el código.

```
[ ACCIÓN HUMANA EQUIVOCADA ] ⟶ [ IMPERFECCIÓN EN EL CÓDIGO ] ⟶ [ COMPORTAMIENTO INCORRECTO EN RUNTIME ]
          (ERROR)                          (DEFECTO / BUG)                          (FALLA)
```

C. Gestión de Incidentes: Severidad vs. Prioridad

Cuando se reporta un defecto, se le asignan dos atributos independientes para coordinar su corrección:

- **Severidad**: Mide el **impacto técnico u operativo** que el defecto causa en la aplicación (fijada técnicamente por el tester).
    - *Niveles de Severidad*: **1 – Bloqueante** (caída del sistema sin *workaround*), **2 – Crítico**, **3 – Mayor**, **4 – Menor**, **5 – Cosmético**.
- **Prioridad**: Indica la **urgencia del negocio** para reparar el defecto en el tiempo (fijada por el Product Owner o Gerente de Proyecto).
    - *Niveles de Prioridad*: **1 – Urgente**, **2 – Alta**, **3 – Media**, **4 – Baja**.
    - *Ejemplo de examen*: Un error de ortografía en el logo de la pantalla principal de un banco tiene **Severidad Cosmética (Baja)** pero **Prioridad Alta/Urgente** por el impacto en la imagen de la empresa.

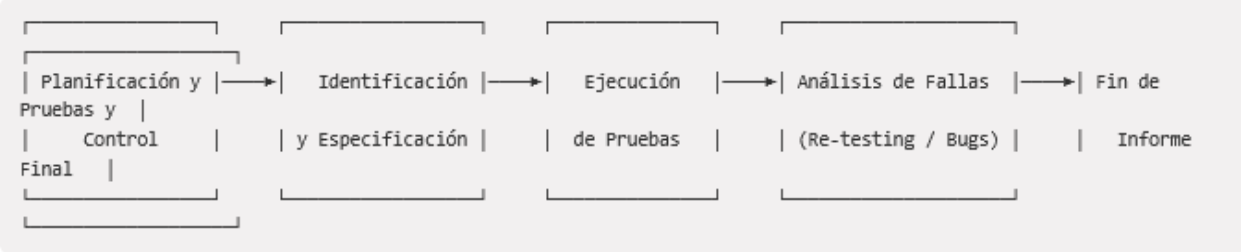
D. La Psicología del Testing e Imposibilidad del Testing Exhaustivo

- **Imposibilidad Práctica**: El testing exhaustivo (probar todas las combinaciones posibles de datos y caminos) es matemáticamente imposible en sistemas reales debido a la explosión combinatoria de entradas y estados. Por ello, las pruebas son un ejercicio de **gestión de riesgos** mediante muestreo representativo.
- **Psicología**: El desarrollador está mentalmente programado para demostrar que el código "funciona"; el tester debe asumir una mentalidad destructiva orientada a "romper el código" para encontrar sus fronteras débiles.

---

2. Proceso y Planificación del Testing de Software

El testing no es una actividad improvisada; es un proceso formal compuesto por etapas secuenciales e iterativas.



A. Etapas del Proceso de Pruebas

1. **Planificación y Control**: Define cómo se incluirá el testing en el proyecto, elabora el Plan de Pruebas, analiza riesgos, recursos y criterios de parada.
2. **Análisis y Diseño (Identificación y Especificación)**: Revisa los requerimientos para verificar su testabilidad y diseña los **Casos de Prueba (CP)**.
3. **Preparación del Ambiente y Ejecución**: Se instancian las bases de datos y servidores, se ejecutan las suites de pruebas y se comparan los resultados reales con los esperados.
4. **Análisis de Fallas y Registro de Incidentes**: Si el resultado real difiere del esperado, se reporta el defecto a los desarrolladores y se realiza el seguimiento (*ping-pong* de corrección).
5. **Cierre e Informe Final**: Se evalúan los criterios de aceptación y parada (p. ej. cero defectos bloqueantes y críticos) y se emite el informe de resultados para la liberación.

B. El Plan de Pruebas (*Test Plan*)

Es el artefacto maestro de la planificación de testing. Se puede elaborar sin tener aún el código fuente, requiriendo únicamente la especificación de requerimientos. Define obligatoriamente:

- Riesgos y objetivos del testing.
- Estrategia de pruebas (niveles, tipos y técnicas aplicables).
- Recursos humanos, herramientas de automatización y calendario.
- Criterios de entrada, criterios de aceptación y criterios de parada (*exit criteria*).

C. Ambientes de Testing (Entornos)

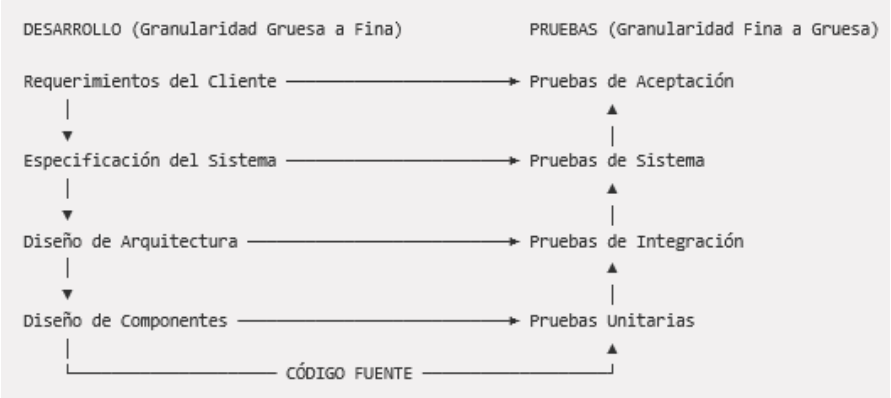
Para evitar interferencias y garantizar la validez de las pruebas, se exige la separación estricta de entornos de hardware y software:

- **Desarrollo**: Entorno de trabajo individual de los programadores donde se realizan las pruebas unitarias e integración inicial.
- **Testing / Pruebas**: Ambiente exclusivo utilizado por los testers para la ejecución de pruebas de sistema. **Los desarrolladores NO deben tener acceso de modificación** sobre este entorno.
- **Pre-producción / Staging**: Réplica exacta de las características del servidor de producción. Se utiliza para ejecutar las pruebas de aceptación y validar requerimientos no funcionales.
- **Producción**: Configuración real donde operan los usuarios finales. **Bajo ningún punto de vista se deben realizar pruebas activas sobre este entorno** por las graves consecuencias operativas que conlleva.

---

3. Niveles de Prueba (Niveles de Granularidad)

Los **niveles de prueba** determinan la granularidad o el foco del software que se está evaluando. Se estructuran según el **Modelo en V**, el cual demuestra que los niveles de prueba se ejecutan en **orden inverso de granularidad** respecto al proceso de desarrollo.



1. Pruebas Unitarias (Unit Testing)

- **Foco**: Evalúa el componente o unidad de código funcional más pequeña del diseño de forma aislada (un método, función o clase).
- **Características**:
    - Realizadas directamente por el **desarrollador** en el ambiente de desarrollo.
    - Exigen acceso total al código fuente (*Caja Blanca*).
    - Apoyadas en *frameworks* de automatización (ej. JUnit, NUnit, pytest).
    - Los errores se reparan de inmediato sobre la marcha, habitualmente sin constancia oficial de reporte.
    - Forman parte del compromiso de la **Definition of Done (DoD)**.

2. Pruebas de Integración (Integration Testing)

- **Foco**: Verifica la interacción y comunicación correcta a través de las interfaces entre dos o más componentes o módulos que ya han superado las pruebas unitarias.
- **Estrategias de Integración Incremental**:
    - **Top-Down (De arriba hacia abajo)**: Se integran primero los módulos de control superior. Requiere el uso de programas auxiliares sustitutos llamados **Stubs** (representantes o simuladores de módulos inferiores no construidos).
    - **Bottom-Up (De abajo hacia arriba)**: Se integran primero los módulos de más bajo nivel o servicios. Requiere el uso de programas auxiliares llamados **Drivers** (controladores que invocan a los módulos probados).
    - **Sandwich / Combinada**: Mezcla ambos enfoques para probar el núcleo del sistema con mayor rapidez.
- **Resultado del Nivel**: Genera el paquete compilado (*build*) que se despliega en el entorno de pruebas.

3. Pruebas de Sistema (System Testing / Version Testing)

- **Foco**: Evalúa el producto ejecutable integrado de forma global para determinar si opera satisfactoriamente como un todo.
- **Características**:
    - Ejecutadas por un **equipo independiente de testing (testers)** en el ambiente de pruebas.
    - Valida tanto requerimientos funcionales como no funcionales (desempeño, seguridad, estrés).
    - El entorno debe imitar lo más fielmente posible al entorno productivo real.

4. Pruebas de Aceptación (Acceptance Testing)

- **Foco**: Evalúa el sistema desde la perspectiva del negocio para determinar si se ajusta a las necesidades del cliente y decidir su despliegue comercial.
- **Característica Crítica de Examen**: **Su objetivo primordial NO es encontrar defectos, sino establecer confianza** en el software para que el cliente proceda a su aceptación formal y firma del contrato.
- **Modalidades de Aceptación**:
    - **Pruebas Alfa**: Pruebas conducidas por los usuarios/clientes en el sitio del desarrollador (*in situ*).
    - **Pruebas Beta**: Lanzamiento de una versión preliminar a un grupo representativo de usuarios finales reales en sus propios ambientes operativos para recibir retroalimentación previa a la venta general.

4. Tipos de Pruebas (Según la Dimensión o Atributo Evaluado)

A diferencia de los niveles (que indican *dónde/cuándo* se prueba), los **tipos de prueba** definen *qué atributo específico* se está verificando.

A. Testing Funcional ("Qué hace el sistema")

Se concentran en evaluar si las funciones y características de la aplicación responden a las especificaciones y procesos de negocio.

- **Basado en Requerimientos**: Verificación directa contra la ERS o los Criterios de Aceptación de las User Stories.
- **Basado en Procesos de Negocio**: Cobertura de flujos end-to-end navegando múltiples módulos (ej. seleccionar producto → facturar → descontar stock).

B. Testing No Funcional ("Cómo funciona el sistema")

Evalúa las características de calidad no funcionales del software. Son más complejas por su estrecha dependencia del entorno técnico.

- **Pruebas de Performance (Rendimiento)**: Miden los tiempos de respuesta y la estabilidad del sistema bajo un número esperado de usuarios concurrentes.
- **Pruebas de Carga (*Load Testing*)**: Miden el comportamiento de los recursos de hardware (CPU, memoria, base de datos) ante volúmenes altos y continuos de transacciones esperadas.
- **Pruebas de Estrés (*Stress Testing*)**: Someten al sistema a cargas extremas por encima de los límites de diseño para evaluar su comportamiento y su capacidad de **recuperación ante fallas** (*graceful degradation*).
- **Pruebas de Usabilidad, Seguridad, Portabilidad y Mantenibilidad**.

C. Tipos de Pruebas Relacionadas con Cambios (Transversales)

1. **Pruebas de Regresión (*Regression Testing*)**:
    - **Definición**: Re-ejecución selectiva de casos de prueba previamente aprobados sobre áreas no modificadas del software para verificar que la introducción de un cambio o la corrección de un bug no haya generado nuevos defectos imprevistos en otras partes del sistema.
    - **Importancia**: Son las candidatas principales a ser **automatizadas** debido a su ejecución repetitiva en cada nueva versión.
2. **Prueba de Humo (*Smoke Test* / *Sanity Check*)**:
    - **Definición**: Corrida rápida y superficial de un conjunto mínimo de pruebas críticas sobre un nuevo *build* recibido para verificar que la versión sea lo suficientemente estable antes de iniciar un ciclo formal de pruebas de sistema.
    - **Objetivo**: Evitar la pérdida de tiempo del equipo de testing si la build contiene errores groseros o bloqueantes en sus funciones básicas.

5. Cuadros Comparativos de Integración para el Examen

Nivel de Prueba	Foco / Granularidad	Ejecutado por	Entorno	Método de Prueba
Unitaria	Componente aislado (método/clase).	Desarrollador	Desarrollo	Caja Blanca (Código)
Integración	Interfaces e interacción entre componentes.	Desarrollador / Tester	Desarrollo / Build	Caja Blanca / Gris
Sistema	Sistema completo e integrado.	Testers (QA)	Pruebas / Testing	Caja Negra (ERS)
Aceptación	Cumplimiento del negocio y confianza.	Usuario / Cliente	Pre-producción / Staging	Caja Negra / Uso Real

B. Testing Funcional vs. Testing No Funcional

Dimensión	Testing Funcional	Testing No Funcional
Interrogante clave	¿*Qué* hace el sistema?	¿*Cómo* funciona el sistema?
Base de evaluación	Requerimientos funcionales y reglas de negocio.	Requerimientos no funcionales y atributos de calidad.
Dependencia del ambiente	Media / Moderada.	**Extremadamente Alta** (requiere imitar hardware real).
Ejemplos típicos	Pruebas de login, cálculo de impuestos, CRUD.	Performance, Carga, Estrés, Seguridad, Usabilidad.

---

6. Relaciones, Mitos y Trampas Frecuentes de Examen

- **Trampa 1: "Las Pruebas de Aceptación tienen como meta principal encontrar bugs"**:
  - **FALSO (Pregunta típica de parcial)**: El foco principal de las pruebas de aceptación es **establecer la confianza en el sistema** para que el cliente lo valide comercialmente, no descubrir defectos.
- **Trampa 2: Confundir Niveles de Prueba con Tipos de Prueba**:
  - *Nivel*: Responde a *cuándo/dónde* (Unitaria, Integración, Sistema, Aceptación).
  - *Tipo*: Responde a *qué atributo* (Funcional, No Funcional, Regresión, Smoke Test).
- **Trampa 3: "La prueba de regresión se realiza solo sobre la funcionalidad que fue corregida"**:
  - **FALSO**: Probar solo la funcionalidad corregida se denomina **Re-testing** o prueba de confirmación. La **Prueba de Regresión** evalúa las funcionalidades del resto del sistema que *no sufrieron cambios* para garantizar que continúen funcionando sin efectos colaterales.
- **Trampa 4: "El ambiente de desarrollo se puede usar para hacer Pruebas de Sistema"**:
  - **FALSO**: El entorno de sistema exige independencia técnica. El equipo de testing debe trabajar en un ambiente donde los desarrolladores no puedan modificar archivos dinámicamente durante el ciclo de pruebas.

Tema 12: Técnicas y herramientas para probar software

---

1. Definición, Objetivo y Clasificación de las Técnicas de Diseño de Pruebas

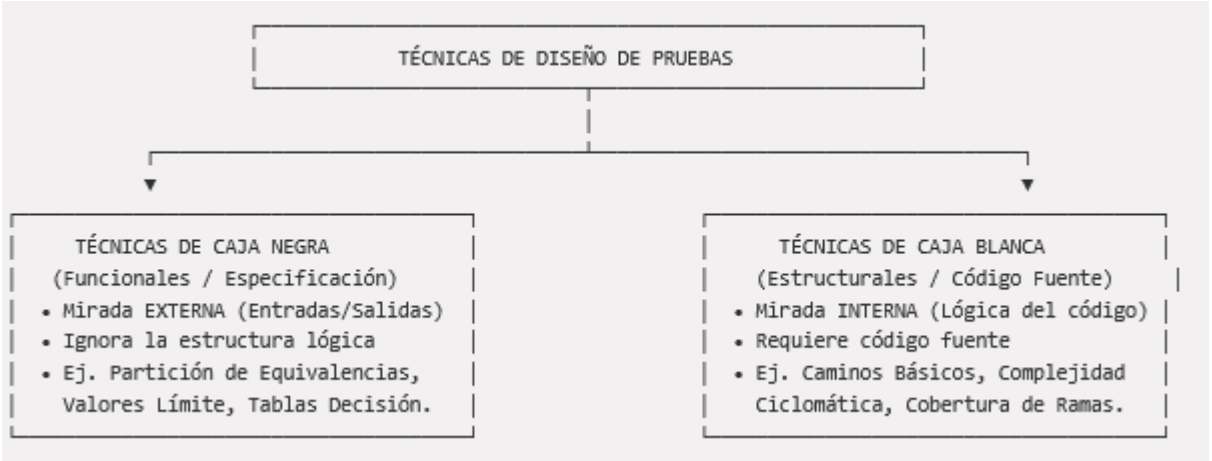
Puesto que el testing exhaustivo (probar el 100% de las combinaciones posibles de datos y caminos) es matemáticamente imposible e ineficiente en tiempo y costo, el **diseño de casos de prueba** requiere un enfoque metódico e inteligente.

Definición Formal

Las **técnicas de diseño de pruebas** son conjuntos de procedimientos, reglas y algoritmos sistemáticos que permiten seleccionar de forma inteligente un **subconjunto mínimo de datos de entrada y escenarios con la mayor probabilidad de descubrir la mayor cantidad posible de defectos**.

Las Dos Estrategias o Enfoques Fundamentales (Foco de Examen):

Según el tipo de información que utiliza el diseñador para derivar los casos de prueba, las técnicas se dividen en dos grandes enfoques complementarios:



1. **Estrategia de Caja Negra (*Black-Box Testing* / Pruebas de Comportamiento)**:
    - Examina la aplicación desde una **visión externa**, considerando al sistema como una "caja opaca".
    - Se derivan los casos de prueba analizando exclusivamente las **especificaciones de requerimientos**, entradas y salidas, sin importar la estructura lógica interna del código fuente.
2. **Estrategia de Caja Blanca (*White-Box Testing* / Pruebas Estructurales o de Caja Transparente)**:
    - Examina la aplicación desde una **visión interna**, basándose en el análisis minucioso de la lógica procedimental, las condiciones, bucles y la estructura del código fuente.

---

2. Métodos y Técnicas de Caja Negra (Basados en Especificaciones y Experiencia)

Las técnicas de caja negra buscan reducir drásticamente el espacio de entrada agrupando datos con comportamientos similares. Se clasifican en dos grupos:

A. Técnicas Basadas en Especificaciones (Formales)

1. Partición de Equivalencia (Equivalence Partitioning)

- **Mecánica**: Divide el dominio de datos de entrada o salida de un campo o condición externa en subconjuntos homogéneos llamados **Clases de Equivalencia**.
- **Premisa de Equivalencia**: Se asume que si un caso de prueba dentro de una clase de equivalencia detecta (o no) un defecto, cualquier otro valor representativo dentro de esa misma clase se comportará de la misma forma.
- **Construcción de Clases (Regla de Examen)**:
    - Se definen **Clases Válidas** (valores aceptados que ejecutan el flujo normal) y **Clases Inválidas** (valores no aceptados que deben disparar manejo de excepciones o mensajes de error).
- **Tabla de Trabajo Estándar UTN**:

Condición Externa (Variable)	Clases de Equivalencia Válidas	Clases de Equivalencia Inválidas
Edad del Usuario (Rango acotado 18 a 65)	CEV1: $18 \leq Edad \leq 65$	CEI1: $Edad < 18$CEI2: $Edad > 65$CEI3: Tipo de dato no numérico

2. Análisis de Valores Límite (Boundary Value Analysis - BVA)

- **Mecánica**: Es una técnica complementaria a la partición de equivalencias. En lugar de seleccionar cualquier valor arbitrario del centro de una clase, selecciona valores específicos en los **extremos, bordes o fronteras de las clases**.
- **Justificación de Errores**: Los desarrolladores cometen estadísticamente la mayor cantidad de errores en los bordes lógicos de las condiciones (p. ej., usar un operador `<` en lugar de un `<=`).
- **Valores a Probar para una Frontera** $[a, b]$:
    - En el límite inferior: El límite exacto (a), justo abajo $(a - 1)$ y justo arriba $(a + 1)$.
    - En el límite superior: El límite exacto (b), justo abajo $(b - 1)$ y justo arriba $(b + 1)$.
    - *Ejemplo*: Para el rango de edad válida \(\), los valores límites exactos son **17, 18, 19** y **64, 65, 66**.

3. Tablas de Decisión (*Decision Tables*)

- **Mecánica**: Herramienta ideal cuando la lógica del sistema está fuertemente basada en combinaciones complejas de condiciones de entrada y reglas de negocio del tipo `if-then-else`.
- **Estructura**: Una matriz que lista en las filas las **Condiciones** (entradas) y las **Acciones** (salidas), y en las columnas las distintas **Reglas de Negocio** combinatorias. Cada regla se transforma directamente en un Caso de Prueba.

4. Transición de Estados / Máquinas de Estado Finito (FSM)

- **Mecánica**: Representa el sistema mediante estados y transiciones gatilladas por eventos. Permite derivar casos de prueba evaluando tres niveles de cobertura: **Cobertura de Estados**, **Cobertura de Transiciones** y **Cobertura de Pares de Transiciones**.
- **Uso**: Ideal para entidades con ciclo de vida (ej. pedido en estado *Pendiente* → *Pagado* → *Enviado* → *Entregado*).

5. Matriz CRUD

- **Mecánica**: Evalúa sistemáticamente las operaciones de **Crear, Leer, Actualizar y Eliminar** (*Create, Read, Update, Delete*) sobre las entidades principales del dominio del sistema.

B. Técnicas Basadas en la Experiencia (Informales / Pragmáticas)

1. **Adivinanza de Defectos (*Error Guessing*)**:
    - Técnica intuitiva donde el tester utiliza su experiencia previa, memoria de errores pasados e intuición para elaborar una lista de situaciones propensas a falla y diseñar pruebas orientadas a exponerlas.
2. **Testing Exploratorio (*Exploratory Testing*)**:
    - Estrategia interactiva donde el **diseño y la ejecución de las pruebas ocurren de forma simultánea**. El tester explora el software en ejecución, aprende sobre su comportamiento y utiliza su creatividad para generar nuevas pruebas en tiempo real.

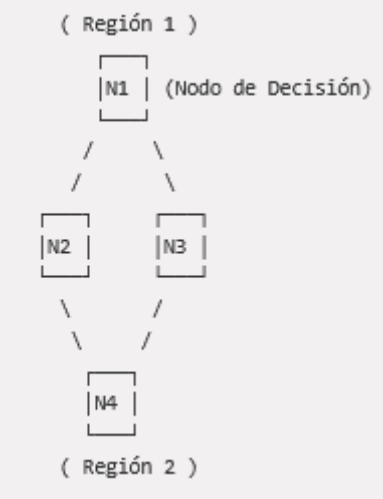
---

3. Métodos y Técnicas de Caja Blanca (Basados en Código Fuente)

Las técnicas de caja blanca analizan la estructura interna del código para garantizar niveles específicos de **cobertura de código (*Code Coverage*)**.

Prueba de Ruta o Camino Básico (Basic Path Testing - McCabe)

Es la técnica de caja blanca por excelencia. Utiliza la estructura lógica del programa representada en un **Grafo de Flujo de Control** (compuesto por nodos de decisión y aristas) para calcular la **Complejidad Ciclomática (M)**.



Cálculo de la Complejidad Ciclomática (M):

Proporciona una medida cuantitativa de la complejidad lógica del algoritmo y define el **número mínimo de caminos independientes** que deben ejecutarse para probar todas las sentencias del código al menos una vez. Se calcula por dos fórmulas equivalentes:

1. **Fórmula por Aristas y Nodos**:
2. $$M = E - N + 2P$$
3. *Donde E es el número de aristas (flechas), N es el número de nodos y P es el número de componentes conexos (habitualmente $P = 1$).*
4. **Fórmula por Regiones Cerradas**:
5. $$M = NúmerodeRegionesCerradas + 1$$
6.

Evaluación del Riesgo según la Complejidad Ciclomática (M):

- $1a10$: Código simple, sin mucho riesgo.
- $11a20$: Más complejo, riesgo moderado.
- $21a50$: Muy complejo, programa de alto riesgo.
- > 50: Programa **no testeable**, muy alto riesgo (requiere refactorización obligatoria).

Niveles de Cobertura de Código (*Code Coverage*)

Definen el grado de exhaustividad con el que la suite de pruebas ejercita las estructuras del código fuente:

- **Cobertura de Sentencias / Instrucciones (*Statement Coverage*)**: Porcentaje de líneas o sentencias ejecutadas al menos una vez. Es el nivel de cobertura más básico e insuficiente.
- **Cobertura de Decisión / Ramas (*Decision / Branch Coverage*)**: Garantiza que cada decisión lógica (como un `if`) haya sido evaluada en su rama Verdadera (`True`) y en su rama Falsa (`False`).
- **Cobertura de Condición (*Condition Coverage*)**: Evalúa individualmente cada sub-condición booleana dentro de una expresión compuesta.
- **Cobertura Múltiple / de Caminos Completo**: Evalúa todas las combinaciones posibles de caminos independientes.

---

4. Herramientas de Testing de Software

Las herramientas de software automatizan y apoyan el diseño, la ejecución y el seguimiento de las pruebas. Se clasifican operativamente en dos grandes familias:

A. Clasificación por Modo de Análisis

1. **Herramientas de Análisis Estático**: Analizan artefactos o código fuente sin ejecutar la aplicación.
    - *Ejemplos*: Linters de código (SonarQube, ESLint), analizadores de complejidad ciclomática (McCabeTest) y herramientas de análisis de requerimientos.
2. **Herramientas de Análisis Dinámico**: Interactúan con la aplicación durante su ejecución real.

B. Clasificación por Función en la Industria

Tipo / Categoría de Herramienta	Función Principal	Ejemplos de Herramientas Reales
Gestión y Trazabilidad	Administración de Casos de Prueba, ejecución de ciclos y Matriz de Trazabilidad.	TestLink, Jira + Xray, Zephyr, Azure DevOps.
Frameworks de Pruebas Unitarias	Automatización e inspección de componentes a nivel de código fuente (TDD).	JUnit, NUnit, PyTest, TestNG, Jasmine.
Automatización de UI / Aceptación	Simulación de acciones del usuario sobre la interfaz gráfica y navegadores web.	Selenium IDE/WebDriver, Cypress, Watir, Appium.
Pruebas de Performance y Carga	Generación de peticiones masivas para medir tiempos de respuesta e infraestructura.	Apache JMeter, Gatling, Locust, LoadRunner.
Métricas de Cobertura	Instrumentación de código para calcular el porcentaje de cobertura de sentencias.	JaCoCo, Cobertura, Coverage.py.

---

5. Cuadro Comparativo: Técnicas de Caja Negra vs. Técnicas de Caja Blanca

Criterio / Dimensión	Técnicas de Caja Negra	Técnicas de Caja Blanca
Base de Información	Especificación de Requerimientos y Casos de Uso.	Código fuente, pseudocódigo o diagramas de flujo.
Perspectiva de Evaluación	Externa (Comportamiento del sistema ante entradas/salidas).	Interna (Lógica procedimental, caminos y bucles).
Nivel de Aplicación Típico	Pruebas de Sistema y Pruebas de Aceptación.	Pruebas Unitarias y Pruebas de Integración.
Perfil del Ejecutor	Testers de QA, Usuarios y Analistas de Negocio.	Desarrolladores de Software.
Técnicas Representativas	Partición de Equivalencias, Valores Límite, Tablas de Decisión, Transición de Estados.	Caminos Básicos (McCabe), Cobertura de Sentencias, Ramas y Condiciones.
Métrica de Suficiencia	Porcentaje de requerimientos o historias cubiertas.	Porcentaje de Cobertura de Código (*Code Coverage*).

---

6. Relaciones, Mitos y Trampas Frecuentes de Examen

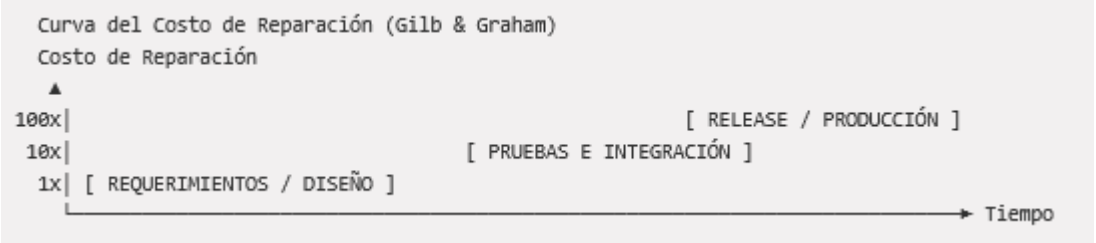
- **Trampa 1: "Las pruebas de caja negra y caja blanca son excluyentes y se debe elegir una sola"**:
  - **FALSO (Pregunta típica de examen)**: Son **técnicas estrictamente complementarias**. La combinación de ambas es la única forma de lograr un testing efectivo, ya que atacan distintas clases de errores.
- **Trampa 2: "Lograr un 100% de cobertura de código (Caja Blanca) garantiza que el software está libre de errores"**:
  - **FALSO**: La prueba de caminos no asegura que el programa responda adecuadamente a las especificaciones de negocio, ni detecta funciones o caminos lógicos faltantes.
- **Trampa 3: Confundir la Fórmula de Complejidad Ciclomática (M)**:
  - Recordar las dos vías de cálculo oficial: $M = E - N + 2P$ o $M = Regionescerradas + 1$. Si el examen presenta un grafo simple, ambas fórmulas deben arrojar exactamente el mismo número.

Tema 13: Técnicas y herramientas para revisiones técnicas del software

---

1. Definición, Objetivos y Naturaleza Estática (Peer Reviews)

Las **Revisiones Técnicas** (o *Peer Reviews* - revisiones por pares) son un proceso de **Verificación y Validación (V&V) de naturaleza estática**. Consisten en el examen metódico de un artefacto de trabajo realizado por colegas o pares del equipo de desarrollo, con el objetivo principal de **detectar defectos y corregirlos en las etapas más tempranas del ciclo de vida**.



A. Principios y Características Clave:

- **Naturaleza Estática**: No requieren ejecutar el software en una computadora.
- **Universalidad**: Se puede inspeccionar **cualquier representación legible del software** (Especificación de Requerimientos - ERS, modelos de diseño UML, código fuente, planes de prueba, manuales de usuario y configuraciones).
- **Axioma de Cátedra**: *"El trabajo técnico necesita ser revisado por la misma razón que los lápices necesitan gomas: errar es humano"*. Ciertas clases de errores se le pasan por alto más fácilmente a quien los origina que a una tercera persona.
- **Origen Histórico**: Introducidas formalmente en 1976 por **Michael Fagan** en IBM, adaptando las inspecciones rígidas de hardware al software para detectar defectos lo más cerca posible de su punto de origen.

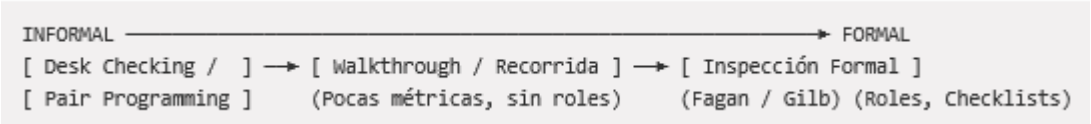
B. Lo que las Revisiones Técnicas "SON" y "NO SON" (Pregunta Clásica de Parcial):

SON	NO SON
• La forma más barata y efectiva de encontrar fallas.	• Utilizadas para buscar o diseñar soluciones a las fallas durante la reunión.
• Una vía para proveer métricas útiles al proyecto.	• Usadas para obtener la aprobación comercial de un entregable.
• Un medio para promover el conocimiento cruzado en el equipo.	• Usadas para evaluar el desempeño o rendimiento individual de las personas.
• Un método probado para mejorar la calidad del producto.	

---

2. Espectro de Formalidad en las Revisiones

Las revisiones técnicas se ubican dentro de un espectro que va desde enfoques altamente informales hasta procesos rigurosamente estructurados:



A. Revisiones Informales (Ej. Walkthrough / Recorrida)

- **Definición**: Técnica de análisis estático donde un diseñador o programador guía a los miembros de su equipo y partes interesadas a través de un producto de software, mientras los participantes formulan preguntas y señalan posibles errores o violaciones de estándares.
- **Atributos**: Poca o ninguna preparación previa, proceso informal, sin control rígido del proceso y con **mínima o nula recolección de métricas**.
- **Objetivos**: Mínima sobrecarga administrativa, capacitación de los desarrolladores y rápido retorno de retroalimentación. Es uno de los métodos informales más elegidos en marcos ágiles al cierre de cada iteración.

B. Revisiones Formales (Inspecciones de Fagan / Gilb)

- **Definición**: Proceso sumamente disciplinado y planeado de evaluación que requiere un conjunto de **roles obligatorios**, uso de **listas de chequeo (*checklists*)**, registro de métricas cuantitativas y la emisión de un reporte oficial de revisión.

---

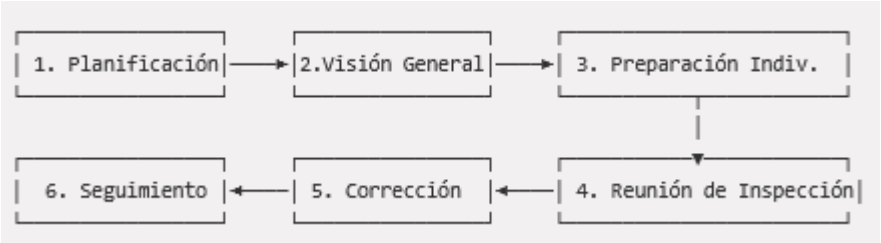
3. El Proceso de Inspección Formal (Fagan / Gilb)

A. Los 5 Roles Participantes (Obligatorios)

1. **Autor**: Creador o encargado del mantenimiento del producto a inspeccionar. Inicia el proceso, entrega el artefacto al moderador, reporta el tiempo de retrabajo y es el **único responsable de corregir los defectos** encontrados.
2. **Moderador**: **Líder de la inspección**. Planifica la revisión, selecciona los roles junto al autor, distribuye el material a los inspectores con al menos 48 horas de anticipación, coordina la reunión asegurando una conducta profesional y realiza el seguimiento de las correcciones.
3. **Anotador / Secretario**: Registra formalmente todos los hallazgos y defectos confirmados durante la sesión y elabora el reporte de la revisión.
4. **Lector**: Lee o parafrasea el producto línea por línea o sección por sección durante la reunión para mantener la concentración colectiva y evitar dispersiones.
5. **Inspector**: Cada uno de los miembros del equipo que examina el producto de forma crítica e individual **antes de la reunión**, utilizando la lista de chequeo y registrando su tiempo de preparación.

Regla de Roles: Aunque en equipos pequeños algunas personas pueden asumir más de un rol (ej. moderador o inspector como lector), **el Autor NUNCA debe actuar como Moderador ni como Anotador**.

B. Las 6 Etapas del Proceso de Inspección Formal



1. **Planificación**: El moderador asigna el equipo, prepara los materiales, la lista de chequeo y fija el calendario.
2. **Visión General (*Overview*)**: El autor presenta brevemente el contexto y antecedentes del producto al equipo.
3. **Preparación Individual**: Cada inspector analiza el artefacto de forma independiente para encontrar potenciales defectos y contabiliza sus horas de preparación.
4. **Reunión de Inspección**: El equipo se reúne para revisar el producto, filtrar falsos positivos y registrar las fallas reales. Se emite la **decisión sobre el producto**:
    - *Aceptar el producto sin modificaciones*.
    - *Aceptar el producto provisoriamente* (el autor corrige fallas menores y el moderador verifica sin necesidad de convocar otra reunión).
    - *Rechazo del producto* (fallas graves que obligan al autor a reparar el código y realizar una re-inspección completa).
5. **Corrección / Retrabajo**: El autor resuelve los defectos registrados en la lista de pendientes.
6. **Seguimiento (*Follow-up*)**: El moderador verifica la correcta aplicación de las soluciones y recopila las métricas finales del proceso.

C. Restricciones Operativas y Tasas de Lectura (Límites Físicos)

Para que la inspección no pierda efectividad por fatiga mental, se imponen los siguientes límites estrictos:

- **Duración Máxima de la Reunión**: **2 horas como máximo**.
- **Tamaño Máximo por Inspección**: **250 Líneas de Código (LOC)** o **280 líneas de texto/documentación**.
- **Velocidades Sugeridas de Procesamiento**:
    - *Preparación*: 100 LOC/hora (código) / 140 líneas de texto/hora (documentos).
    - *Inspección en reunión*: 125 LOC/hora (código) / 140 líneas de texto/hora (documentos).

---

4. Métricas de Revisiones Técnicas (Cátedra UTN)

Las inspecciones proporcionan datos cuantitativos fundamentales para el modelo de calidad:

- **Densidad de Defectos**:
    - $DensidaddeDefectos = \frac{TotaldeDefectosEncontrados}{TamañodelArtefacto(LOCoPáginas)}$
- **Total de Defectos Encontrados**:
    - $TotalDefectos = DefectosMayores + DefectosMenores$
- **Esfuerzo de la Inspección**: \[\text{Esfuerzo}*{\text{Total}}* = *text{Esfuerzo}*{\text{Planificación}} + \text{Esfuerzo}*{*text{Preparación}}* + *text{Esfuerzo}*{\text{Reunión}} + \text{Esfuerzo}_{\text{Retrabajo}}\]
- **Esfuerzo por Defecto**:
    - $EsfuerzoporDefecto = \frac{EsfuerzoTotaldeInspección}{TotaldeDefectosEncontrados}$
- **Porcentaje de Re-inspecciones**:
    - $\%Re - inspecciones = \left(\frac{CantidaddeRe-inspecciones}{CantidadTotaldeInspecciones}\right) \times 100$

---

5. Lineamientos y Reglas de Oro para Revisiones Técnicas

1. **Revisar el producto... NO al productor**: Enfocarse en la calidad técnica del artefacto sin juzgar a la persona.
2. **Fijar una agenda y cumplirla estrictamente**: Mantener el tiempo acotado.
3. **Limitar el debate y las discusiones paralelas**: Si un punto genera controversia, se anota el problema y se continúa.
4. **Enunciar las áreas de problemas, pero NO tratar de resolver el defecto en la reunión**: La sesión es para *detectar* errores, no para rediseñar ni corregir el código.
5. **Tomar notas escritas minuciosas**: Mantener registro formal de las observaciones.
6. **Limitar el número de participantes (3 a 5 personas) e insistir en la preparación previa obligatoria**.
7. **Desarrollar y utilizar listas de chequeo (*checklists*)**: Guiar la revisión hacia fallas históricas o recurrentes.

---

6. Cuadro Comparativo: Walkthrough (Informal) vs. Inspección (Formal)

Dimensión / Criterio	Walkthrough / Recorrida (Informal)	Inspección (Formal)
Proceso y Estructura	Informal, sin secuencia rígida de pasos.	**Proceso Formal**, disciplinado en 6 etapas.
Roles Participantes	No definidos formalmente; el autor conduce la sesión.	**5 Roles Obligatorios**: Autor, Moderador, Anotador, Lector e Inspector.
Uso de Checklists	Opcional o no utilizado.	**OBLIGATORIO**; guía la lectura previa.
Preparación Previa	Poca o ninguna preparación.	**Exigida y contabilizada en horas**.
Medición y Métricas	No toma métricas sistemáticas.	**Avanzada**: Mide densidad, esfuerzo y tasas.
Objetivo Principal	Mínima sobrecarga, capacitación y rápido retorno.	Detectar y remover el 100% de los defectos eficientemente.

Tema 14: Testing en ambientes ágiles

---

1. Definición, Cambio de Paradigma y Manifiesto del Testing Ágil

El **Testing en Ambientes Ágiles (*Agile Testing*)** no es un conjunto de herramientas ni un nivel de prueba aislado, sino una **filosofía y disciplina técnica continua** alineada con los valores del Manifiesto Ágil.

A. El Cambio de Paradigma Fundamental:

- **En el Enfoque Tradicional (Cascada)**: El testing es una **fase tardía y cerrada** que ocurre al final del ciclo de vida, orientada principalmente a la **detección y reporte de fallas** (*"encontrar bugs"*) sobre el producto ya construido.
- **En el Enfoque Ágil**: El testing es una **actividad transversal y continua** que acompaña todo el proceso de desarrollo. Su objetivo central no es encontrar errores al final, sino **prevenir la introducción de defectos (*prevent bugs*) desde el momento cero**.

B. El Manifiesto del Testing Ágil (*Testing Manifesto*):

Propone cinco transformaciones culturales y conceptuales en la forma de gestionar la calidad:

Enfoque Tradicional (Menor Valor)	Enfoque Ágil (Valorado Preferentemente)
Pruebas al final del desarrollo	**Pruebas continuas a lo largo de todo el ciclo**
Encontrar defectos (*Finding bugs*)	**Prevenir defectos (*Preventing bugs*)**
Verificar especificaciones / funcionalidades	**Entender y clarificar qué se está probando con el negocio**
Tratar de "romper el sistema"	**Ayudar a construir el mejor sistema posible**
El tester es el único responsable de la calidad	**Responsabilidad colectiva de todo el equipo por la calidad (*Whole-Team Approach*)**

---

2. Principios y Prácticas Clave del Testing Ágil

A. Los 9 Principios del Testing Ágil (Cátedra UTN / Crispin & Gregory):

1. **El testing se mueve hacia adelante**: Se prueba desde la definición de requerimientos, antes de escribir el código fuente.
2. **El testing NO es una fase**: Es una actividad continua embebida en cada día del Sprint.
3. **Todos hacen testing**: Desarrolladores, testers y Product Owner colaboran en la calidad.
4. **Reducir la latencia de la retroalimentación (*Feedback Loop*)**: Detectar fallas en minutos o segundos mediante integración continua.
5. **Las pruebas representan expectativas**: Los casos de prueba automatizados actúan como la especificación viva del comportamiento esperado.
6. **Mantener el código limpio y corregir defectos rápido**: Cero tolerancia a acumular deuda técnica o fallas no resueltas.
7. **Reducir la sobrecarga de documentación**: Priorizar pruebas ejecutables automatizadas frente a extensos documentos estáticos de casos de prueba.
8. **Las pruebas son parte del "Terminado" (*Done*)**: Una Historia de Usuario no está terminada si sus pruebas no han sido automatizadas y aprobadas.

9. **De probar al final a ser "Conducido por Pruebas"**: Transición hacia técnicas donde la prueba diseña la arquitectura (TDD/ATDD).

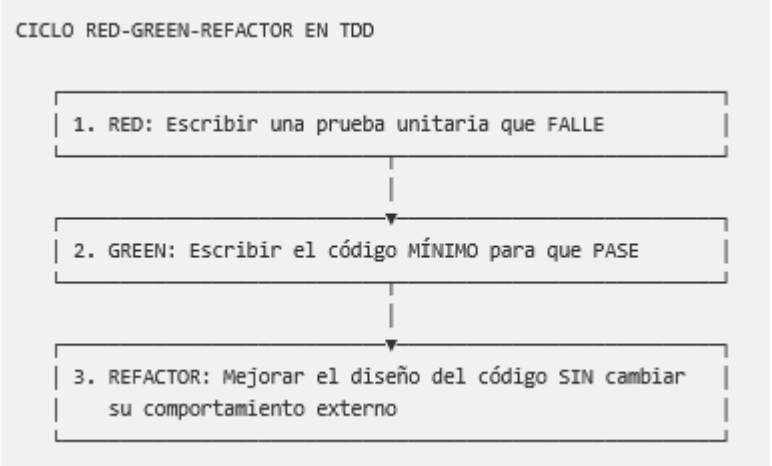
B. Las 6 Prácticas Concretas en Equipos Ágiles:

1. Pruebas de unidad e integración automatizadas.
2. Desarrollo Conducido por Pruebas (**TDD**).
3. Pruebas de regresión a nivel de sistema totalmente automatizadas.
4. Desarrollo Conducido por Pruebas de Aceptación (**ATDD**).
5. Pruebas Exploratorias manuales e inteligentes.
6. **Control de versión de las pruebas junto con el código fuente** en el mismo repositorio.

---

3. Técnicas de Desarrollo Conducido por Pruebas: TDD, ATDD y BDD

En el agilismo, la automatización de pruebas se transforma en un **mecanismo de diseño de software y clarificación de requerimientos**.



A. Desarrollo Conducido por Pruebas (Test-Driven Development - TDD)

- **Definición**: Técnica de diseño e ingeniería de software donde el desarrollador escribe primero la prueba unitaria automatizada antes de escribir el código de producción.
- **El Corazón de TDD: Ciclo Red-Green-Refactor**:
    1. **RED (Rojo)**: Se escribe una pequeña prueba unitaria que falla porque la funcionalidad aún no existe en el sistema.
    2. **GREEN (Verde)**: Se implementa la cantidad mínima e indispensable de código para lograr que la prueba pase con éxito.
    3. **REFACTOR (Refactorización)**: Se limpia el código y las pruebas eliminando duplicación, mejorando nombres y estructura sin alterar el comportamiento observable.
- **Las 3 Leyes de TDD (Robert C. Martin / UTN)**:
    1. *No escribir código de producción hasta que se haya escrito primero una prueba unitaria que falle*.
    2. *No escribir más de una prueba unitaria de la necesaria para fallar (un fallo de compilación es un fallo)*.
    3. *No escribir más código de producción del necesario para hacer pasar la prueba que falla*.
- **Beneficios de TDD**: Genera código modular con alta cohesión y bajo acoplamiento, otorga una red de seguridad para refactorizar sin miedo y garantiza el 100% de cobertura de prueba en componentes críticos.

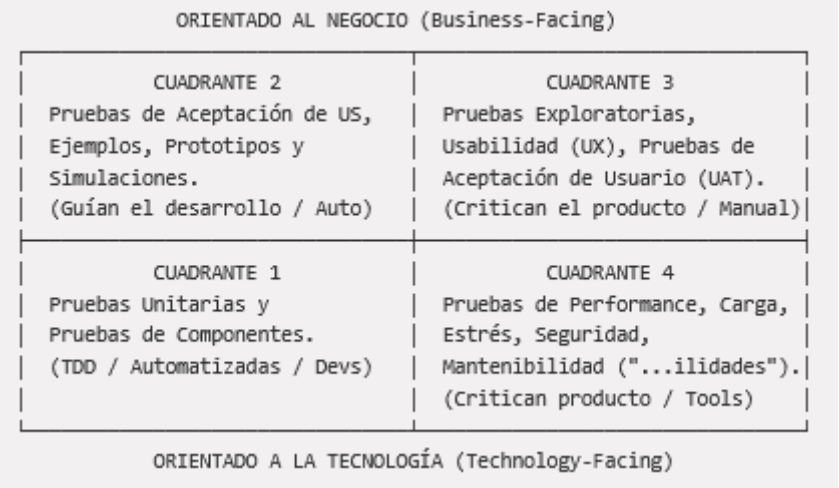
B. Desarrollo Conducido por Pruebas de Aceptación (ATDD / STDD)

- **Definición**: Extensión de TDD orientada al negocio. Consiste en redactar los **criterios y pruebas de aceptación con ejemplos ejecutables** antes de comenzar a codificar la Historia de Usuario.
- **Dinámica de Colaboración (La "Tríada" / *Three Amigos*)**: Participan activamente el **Product Owner / Cliente**, el **Desarrollador** y el **Tester** para acordar los escenarios de prueba en lenguaje del dominio.
- **BDD (Behavior-Driven Development)**: Utiliza un lenguaje estructurado formal en lenguaje natural (Domain Specific Language como *Gherkin*) con la sintaxis **Dado... Cuando... Entonces...** (*Given... When... Then...*) para automatizar pruebas de comportamiento alineadas al negocio.

---

4. Los Cuadrantes del Agile Testing (Lisa Crispin & Janet Gregory)

Los **Cuadrantes del Agile Testing** (adaptados del modelo de Brian Marick) son una matriz de clasificación que ayuda al equipo a planificar e identificar qué tipos de pruebas se necesitan, cuáles deben automatizarse y qué roles intervienen.

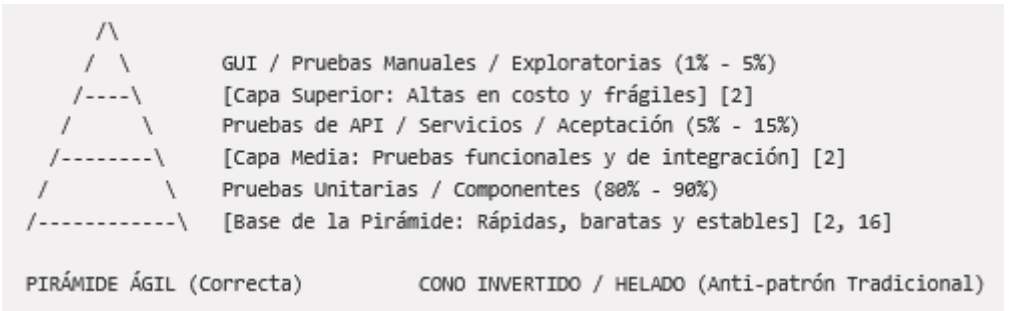


1. **Cuadrante 1 (Q1 - Tecnología / Soporte al Equipo)**:
    - **Contenido**: Pruebas Unitarias y de Componentes.
    - **Características**: Totalmente **automatizadas**, asociadas a TDD y ejecutadas por los desarrolladores.
2. **Cuadrante 2 (Q2 - Negocio / Soporte al Equipo)**:
    - **Contenido**: Pruebas funcionales de Historias de Usuario, ejemplos, pruebas de aceptación (ATDD) y prototipos.
    - **Características**: Guían el desarrollo y son **automatizadas y manuales**.
3. **Cuadrante 3 (Q3 - Negocio / Critica el Producto)**:
    - **Contenido**: Pruebas Exploratorias, Usabilidad (UX), pruebas de escenarios y pruebas de aceptación del usuario final (UAT).
    - **Características**: Principalmente **manuales**; se enfocan en la experiencia real del usuario.
4. **Cuadrante 4 (Q4 - Tecnología / Critica el Producto)**:
    - **Contenido**: Pruebas de Rendimiento, Carga, Estrés, Seguridad, Concurrencia y atributos no funcionales ("...ilidades").
    - **Características**: Apoyadas en **herramientas de automatización especializadas** (ej. JMeter, ZAP).

---

5. La Pirámide de Automatización de Pruebas (Mike Cohn / Agile Automation Pyramid)

La **Pirámide de Testing Ágil** establece la proporción idónea y la inversión económica adecuada que un equipo debe realizar al automatizar sus pruebas.



A. Capas de la Pirámide Ágil:

1. **Base de la Pirámide (80% - 90% del total)**: **Pruebas Unitarias Automatizadas**.
    - *Propiedades*: Extremadamente rápidas de ejecutar, baratas de escribir y mantener, altamente deterministas y con el mayor Retorno de Inversión (ROI).
2. **Capa Media (5% - 15% del total)**: **Pruebas de Capa de Servicios / API / Integración / ATDD**.
    - *Propiedades*: Verifican la lógica del negocio y las reglas de integración sin pasar por la interfaz gráfica.
3. **Cúpula de la Pirámide (1% - 5% del total)**: **Pruebas de Interfaz Gráfica (GUI) / End-to-End y Pruebas Manuales Exploratorias**.
    - *Propiedades*: Lentas de ejecutar, muy costosas de mantener y propensas a romperse ante cambios menores de UI (*frágiles*).

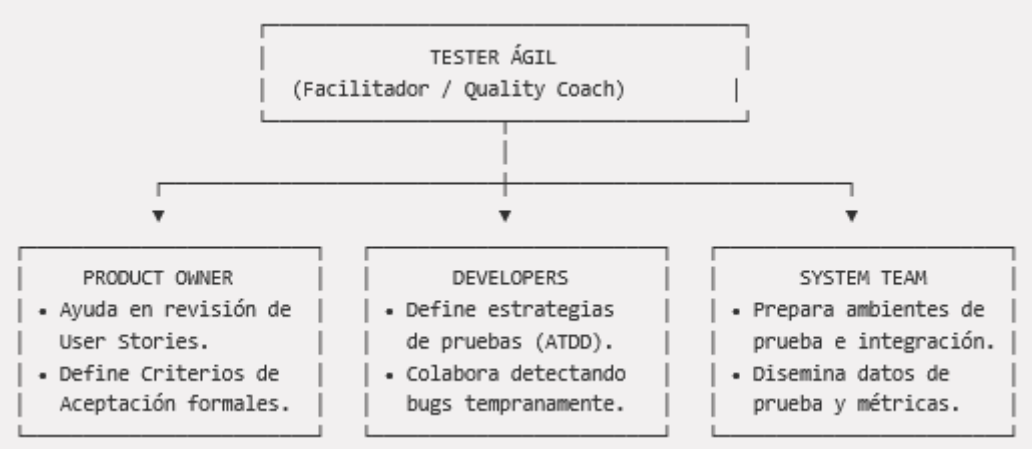
B. El Anti-Patrón: El "Cono Invertido de Helado" (*Inverted Ice Cream Cone*):

Es el error típico de equipos tradicionales que intentan ser ágiles sin transformar sus prácticas de ingeniería: automatizan masivamente a nivel de interfaz de usuario (GUI - 80%) y poseen una base delgada de pruebas unitarias (1-5%). Esto paraliza la Integración Continua por los largos tiempos de ejecución y la fragilidad del mantenimiento.

---

6. El Rol del Tester en Equipos Ágiles

En ágil, el rol del tester muta de un "fiscalizador externo que busca culpar" hacia un **consultor y facilitador de calidad interno (*Quality Coach*)**:



- **Colaboración Multidireccional**:
    - *Con el Product Owner*: Colabora durante el refinamiento redactando los Criterios de Aceptación y aclarando dudas de negocio.
    - *Con los Developers*: Trabaja codo a codo en parejas (*pairing*) ayudando a diseñar escenarios de prueba y automatizaciones funcionales.
    - *Con el Equipo en General*: Promueve que el cumplimiento de la **Definition of Done (DoD)** sea estricto para mantener la rama principal siempre desplegable.

---

7. Cuadro Comparativo: Testing Tradicional vs. Testing Ágil

Criterio / Dimensión	Testing Tradicional (Cascada)	Testing en Ambientes Ágiles
Ubicación en el Ciclo	**Fase tardía e independiente** al final del proyecto.	**Actividad continua y paralela** durante todo el Sprint.
Objetivo Principal	Encontrar y reportar defectos (*Detect bugs*).	**Prevenir la ocurrencia de defectos (*Prevent bugs*)**.
Responsabilidad de Calidad	Exclusiva del área/equipo de Testing.	**Responsabilidad colectiva del equipo (*Whole-Team*)**.
Estructura de Automatización	Cono Invertido (Foco en GUI / Testing manual masivo).	**Pirámide Ágil** (80-90% Pruebas Unitarias en la base).
Técnicas Clave	Pruebas de Sistema al final, scripts manuales rígidos.	**TDD, ATDD, BDD, Pruebas Exploratorias y CI/CD**.
Rol del Tester	"Cazador de errores" segregado del desarrollo.	"Quality Coach", colaborador directo de PO y Devs.

---

8. Relaciones, Mitos y Trampas Frecuentes de Examen

- **Trampa 1: "En metodologías ágiles el rol del tester desaparece porque los desarrolladores hacen TDD"**:
  - **FALSO (Pregunta clásica de examen)**: El rol del tester no desaparece; sus habilidades se reorientan a la **visión holística de calidad, automatización de nivel funcional (ATDD), pruebas no funcionales y pruebas exploratorias de alto valor**.
- **Trampa 2: "TDD es una técnica de pruebas para encontrar bugs"**:
  - **FALSO**: TDD es principalmente una **técnica de diseño de arquitectura de software** que utiliza las pruebas unitarias para guiar la construcción de código limpio y desacoplado.
- **Trampa 3: "La pirámide de automatización recomienda automatizar el 80% de las pruebas a nivel de interfaz de usuario (GUI)"**:
  - **FALSO**: Eso corresponde al anti-patrón de Cono Invertido. La Pirámide Ágil establece que el **80-90% de las pruebas automatizadas deben ser unitarias (base)**, dejando solo un 1-5% para la interfaz gráfica.